

Progetto 6: Valutazione e Fine-Tuning di Modelli Linguistici (LLM)

Gruppo di Progetto

5 marzo 2026

1 Introduction

1.1 Problem Description

Il **Progetto 6: Valutazione e Fine-Tuning di Modelli Linguistici (LLM)** ha come obiettivo principale l'analisi comparativa di modelli linguistici pre-addestrati, valutandone le capacità di adattamento a task specifici. Il focus del lavoro risiede nell'esecuzione pratica di diverse strategie di **Fine-Tuning** su modelli distinti, al fine di misurarne quantitativamente le prestazioni post-adattamento e farne emergere, tramite confronto diretto, i rispettivi **punti di forza e di debolezza**.

Nello specifico, il nostro gruppo ha selezionato il task di **Emotion Recognition** (Classificazione delle Emozioni) applicato a testi brevi provenienti dai social media. La rilevanza di questo problema è elevata nell'attuale panorama digitale: la capacità di analizzare automaticamente il "sentiment" e le sfumature emotive (gioia, tristezza, rabbia, ecc.) su piattaforme come Twitter è cruciale per applicazioni di monitoraggio della brand reputation, analisi sociale e customer care automatizzato.

Il pubblico di riferimento include ricercatori NLP, data scientist e sviluppatori di assistenti virtuali empatici. Una soluzione efficace permette di automatizzare l'analisi di grandi moli di dati non strutturati (Big Data) con costi ridotti rispetto all'annotazione manuale.

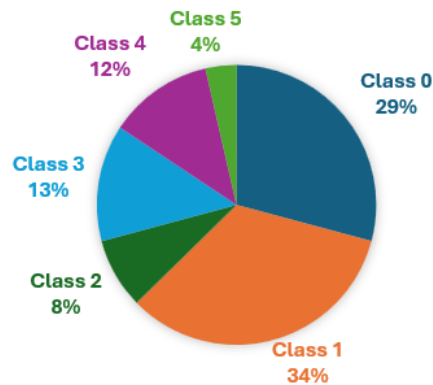
1.2 Proposed Solution

Per affrontare il problema, abbiamo adottato un **approccio sperimentale comparativo**, testando diverse tecniche di fine-tuning su una selezione eterogenea di modelli pre-addestrati. L'obiettivo centrale è stato confrontare l'efficacia delle diverse strategie di adattamento in relazione alle architetture sottostanti **in riferimento al Dataset scelto**:

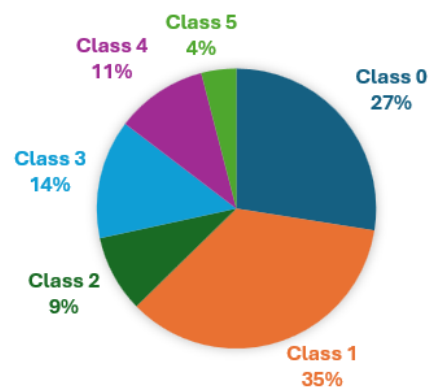
- **Emotion di Hugging Face**: pubblicato da dair-ai, è una raccolta di messaggi di Twitter in inglese annotati con sei emozioni di base: sadness (tristezza), joy (gioia), love (amore), anger (rabbia), fear (paura) e surprise (sorpresa), pensato per compiti di text classification multiclassificazione e ricerca nel riconoscimento delle emozioni nel testo. I dati sono suddivisi nei classici split train, validation e test. Una caratteristica importante evidenziabile dai grafici, è che la distribuzione delle classi non è bilanciata: alcune emozioni (come sadness e joy) tendono ad essere molto più rappresentate rispetto ad altre (come surprise o love), soprattutto nel training

set. Questo sbilanciamento può influenzare l'addestramento dei modelli perché le classi maggioritarie dominano l'apprendimento, mentre quelle minoritarie rischiano di essere trascurate.

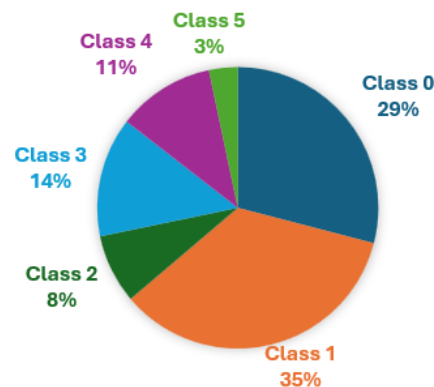
DATA DISTRIBUTION ON TRAINING SET



DATA DISTRIBUTION ON EVALUATION SET



DATA DISTRIBUTION ON TEST SET



La nostra soluzione si articola nei seguenti punti:

- **Modelli Selezionati:** Abbiamo condotto esperimenti su quattro architetture distinte per valutare l'impatto della dimensione e della specializzazione del pre-training:
 - vinai/bertweet-base: Modello domain-specific per Twitter.
 - distilbert/distilbert-base-uncased: Modello distillato (leggero) per testare l'efficienza.
 - google-bert/bert-base-uncased: La baseline standard per il NLP.
 - FacebookAI/roberta-base: Versione ottimizzata di BERT.
- **Approccio Metodologico:** Un confronto sistematico tra diverse metodologie di apprendimento:
 1. **Transfer Learning (Frozen):** Addestramento del solo classificatore mantenendo il corpo del modello congelato.
 2. **Complete Fine-Tuning:** Aggiornamento di tutti i pesi del modello per l'adattamento al task. Si specifica inoltre che, su alcuni modelli, si è scelto di utilizzare anche **LoRA (Low-Rank Adaptation)**, una tecnica PEFT che inietta matrici addestrabili nei layer del modello.
 3. **In-Context Learning (Zero/Few-Shot):** Per testare le capacità native senza alcun aggiornamento dei pesi.
 1. **In-Context Learning:** In seguito all'addestramento del solo classificatore mantenendo il corpo del modello congelato (Alignment classification head), si è utilizzato il metodo di prompt engineering few-shot con diversi gradi.
 2. **Complete Fine-Tuning:** Aggiornamento di tutti i pesi del modello per l'adattamento al task.
- **Sfide Computazionali:** Le principali difficoltà affrontate sono state la gestione della **Context Window** limitata (tipica dei modelli Encoder-Only), che ha imposto vincoli severi sulle strategie di Few-Shot Learning, e la sfida di massimizzare le performance di generalizzazione nonostante il **basso numero di parametri** dei modelli "base" e "distilled" selezionati, richiedendo un'ottimizzazione fine degli iperparametri.

Task Distribution: Il lavoro è stato parallelizzato assegnando a ciascun membro del gruppo la responsabilità completa di una specifica architettura. Ogni componente ha curato l'analisi esplorativa, il pre-processing, il fine-tuning e la valutazione del proprio modello, per poi convergere in una fase finale di confronto dei risultati.

- **[Alexandru Bogdan Nicolescu]:** Responsabile per vinai/bertweet-base.
- **[Pietro Cimino]:** Responsabile per distilbert/distilbert-base-uncased.
- **[Mattia Boffa]:** Responsabile per google-bert/bert-base-uncased.
- **[Alessandro Modelli]:** Responsabile per FacebookAI/roberta-base.

Summary of Achieved Results: TODO

1.3 [extra] Literature Review

In letteratura, il benchmark per il riconoscimento delle emozioni su testo fa spesso riferimento al dataset `dair-ai/emotion`, introdotto originariamente da Saravia et al. (2018)saravia-et al-20 nel lavoro “CARER: Contextualized Affect Representations for Emotion Recognition”. Questo dataset è stato costruito utilizzando hashtag come etichette deboli (distant supervision) per classificare i tweet in categorie emotive fondamentali.

Per quanto riguarda i modelli, lo stato dell’arte su dati provenienti da Twitter è rappresentato da architetture domain-specific come **BERTweet**, presentato da Nguyen et al. (2020) nguyen-et al-2020-bertweet. **BERTweet** utilizza la stessa architettura di BERT-base ma è stato pre-addestrato su un corpus massivo di 850 milioni di tweet inglesi, dimostrando prestazioni superiori rispetto a RoBERTa e XLM-R su task di classificazione testuale, POS tagging e NER in ambito social media.

Tuttavia, approcci di Full Fine-Tuning su questi modelli richiedono risorse computazionali significative. Per mitigare questo problema, Hu et al. (2021) **LoRA** hanno proposto **LoRA (Low-Rank Adaptation)**. Questa tecnica congela i pesi del modello pre-addestrato e inietta matrici di decomposizione di rango ridotto nei layer del Transformer, riducendo drasticamente il numero di parametri addestrabili (fino a 10.000 volte in meno rispetto al fine-tuning completo su GPT-3) senza introdurre latenza in inferenza. Il nostro lavoro dimostra come l’applicazione di LoRA su BERTweet, se ben calibrata, possa eguagliare i metodi tradizionali a una frazione del costo computazionale.

2 Proposed Method

2.1 Solution Choice

Abbiamo scartato l’uso di LLM generativi generalisti (es. GPT-4, Llama-3) per motivi di latenza in inferenza e costi computazionali, preferendo architetture **Encoder-Only** (come **BERTweet** e le varianti di BERT/RoBERTa). I modelli Encoder sono strutturalmente superiori nei task di classificazione pura grazie all’attenzione bidirezionale che cattura l’intero contesto della frase simultaneamente.

Inizialmente è stato eseguito un addestramento del solo classification head che pur essendo computazionalmente leggero e veloce presenta lo svantaggio di congelare il corpo del modello, limitandone drasticamente la capacità di apprendimento su domini complessi al fine di testare la tecnica di In-Context Learning per testare... , scontrandoci tuttavia con i severi limiti imposti dalla context window ristretta tipica di queste architetture. Successivamente, abbiamo testato il Complete Fine-Tuning, il quale aggiorna la totalità dei pesi garantendo un’ottima accuratezza, ma risultando al contempo lento e molto oneroso in termini di memoria.

Per quanto riguarda le tecniche di addestramento e adattamento, abbiamo optato per un confronto a tutto tondo al fine di comprendere appieno i trade-off tra risorse impiegate e prestazioni ottenute. Inizialmente abbiamo valutato il **Transfer Learning** classico (approccio *Feature Extraction*), che pur essendo computazionalmente leggero e veloce presenta lo svantaggio di congelare il corpo del modello, limitandone drasticamente la capacità di apprendimento su domini complessi. Successivamente, per stabilire un limite superiore (*upper bound*) alle performance, abbiamo testato il **Complete Fine-Tuning**, il quale aggiorna la totalità dei pesi garantendo un’ottima accuratezza, ma risultando al contempo lento e molto oneroso in termini di memoria. Per avere un quadro sperimentale completo,

abbiamo esplorato anche l'**In-Context Learning** per testare le capacità native (*zero-shot* e *few-shot*) dei modelli senza effettuare alcun aggiornamento dei pesi, scontrandoci tuttavia con i severi limiti imposti dalla *context window* ristretta tipica di queste architetture. Infine, proprio per superare l'inefficienza e i costi del fine-tuning completo e la rigidità del transfer learning classico, su alcuni modelli si è scelto di implementare **LoRA (Low-Rank Adaptation)**. Questa tecnica permette di adattare efficacemente le rappresentazioni interne del modello al task specifico delle emozioni iniettando e modificando solo poche matrici di peso aggiuntive.

2.2 Methodology for performance measurement

Le prestazioni sono state valutate utilizzando un set di metriche standard per la classificazione multiclasse, selezionate per fornire una visione sia quantitativa che qualitativa:

- **Accuracy:** Rappresenta la frazione di predizioni corrette rispetto al totale dei campioni. Sebbene sia la metrica più intuitiva per una valutazione globale, in questo progetto viene utilizzata solo come riferimento di base, poiché può risultare ingannevole in presenza di dataset sbilanciati (un modello che predice sempre la classe maggioritaria avrebbe comunque un'accuratezza elevata).
- **Macro F1-Score:** È la media armonica tra *Precision* e *Recall*, calcolata indipendentemente per ogni classe e poi mediata (senza pesare per la numerosità delle istanze). Questa è la metrica **critica** per il nostro task a causa del forte sbilanciamento del dataset (le classi *Joy* e *Sadness* sono molto più frequenti di *Surprise* e *Love*). L'uso del *Macro-average* penalizza i modelli che ignorano le classi minoritarie, garantendo che ogni emozione abbia lo stesso peso nella valutazione finale.
- **F1-Score:** È la media armonica tra *Precision* e *Recall*, calcolata indipendentemente per ogni classe e poi mediata (senza pesare per la numerosità delle istanze). Questa è la metrica **critica** per il nostro task a causa del forte sbilanciamento del dataset (le classi *Joy* e *Sadness* sono molto più frequenti di *Surprise* e *Love*).
- **Confusion Matrix:** È una tabella che visualizza il confronto tra le etichette reali (righe) e quelle predette dal modello (colonne). Viene utilizzata per l'**analisi qualitativa** degli errori, permettendo di diagnosticare non solo *quanto* il modello sbaglia, ma *dove* sbaglia (ad esempio, rivelando se l'emozione *Love* viene sistematicamente confusa con *Joy* a causa della sovrapposizione semantica).

3 Experimental Results

3.1 bertweet-base

3.1.1 Demonstration and Technologies

Per garantire la riproducibilità degli esperimenti, il progetto è stato sviluppato utilizzando le seguenti tecnologie e versioni:

- **Environment:** Ambiente di sviluppo locale (Python 3.13.12).

- **Librerie:**

- `transformers` (HuggingFace) per la gestione del modello BERTweet.
- `peft` (HuggingFace) per l'implementazione di LoRA.
- `datasets` e `evaluate` per la gestione dei dati e delle metriche.
- `scikit-learn` per il calcolo delle matrici di confusione.

- **Istruzioni Demo:** Il notebook fornito esegue automaticamente il download del dataset, il caricamento dei pesi LoRA ottimizzati e la valutazione sul validation set, stampando sul notebook le metriche finali.

3.1.2 Results

La configurazione migliore identificata (**LoRA Optimized**) ha ottenuto i seguenti risultati sul Validation Set (Best Epoch):

- **Accuracy:** 94.20%
- **Macro F1-Score:** 91.69%
- **Training Time:** ~ 15 minuti (10 epoche).

Questa configurazione utilizzava un Rango $r = 8$, un Learning Rate di $2e^{-4}$ con *Cosine Scheduler*, e applicava LoRA a **tutti i layer lineari** del trasformatore (`query`, `key`, `value`, `dense`), non solo a quelli di attenzione.

3.1.3 Ablation Study: Analysis of Configurations

Lo studio di ablation è stato condotto in tre fasi distinte per isolare l'impatto dei singoli iperparametri e delle limitazioni architetturali.

3.1.4 1. Impatto del Context Window (In-Context Learning)

Abbiamo valutato l'efficacia dell'aggiunta di esempi (k -shot) nel prompt. I risultati mostrano che aumentare k non migliora le prestazioni a causa della limitata finestra di contesto di BERTweet (128 token). Con $k = 5$, il modello inizia a troncare il prompt, perdendo informazioni cruciali.

Shots (k)	Accuracy	Truncated	Note
0	49.30%	0	Baseline Zero-Shot (Migliore).
1	46.87%	0	Nessun miglioramento significativo.
3	48.40%	0	Inizio saturazione contesto.
5	48.88%	25	Degrado tecnico: 25 esempi su 2000 troncati.

Tabella 1: Ablation su In-Context Learning

3.1.5 2. Tuning del Transfer Learning (Frozen)

Mantenendo il corpo del modello congelato, abbiamo condotto una *Grid Search* completa combinando tre valori di Learning Rate ($LR \in \{1e^{-3}, 1e^{-4}, 1e^{-5}\}$) con due dimensioni di Batch Size ($BS \in \{8, 16\}$), per un totale di 6 configurazioni sperimentali. I risultati indicano che un LR più aggressivo ($1e^{-3}$) è strettamente necessario per addestrare efficacemente la testa di classificazione da zero; tuttavia, l'accuracy massima rimane bloccata attorno al 54%, confermando l'insufficienza del metodo *Frozen* per questo task.

Learning Rate	Batch Size	Accuracy	F1-Score
$1e^{-3}$	16	54.45%	29.9%
$1e^{-4}$	16	50.25%	20.8%
$1e^{-5}$	16	40.85%	14.9%

Tabella 2: Sensibilità al Learning Rate nel Transfer Learning (Batch Size fissato a 16)

3.1.6 3. Configurazione Ottimale di LoRA

Per individuare l'architettura PEFT ideale, abbiamo eseguito una *Grid Search* estensiva testando 36 combinazioni diverse di iperparametri: Rango ($r \in \{8, 16, 32\}$), Moduli Target (solo attenzione vs tutti i layer lineari), Learning Rate ($1e^{-3}, 1e^{-4}, 1e^{-5}$) e Batch Size (8, 16). I dati dimostrano che targettare **tutti i moduli lineari** (all-linear) con un rango contenuto ($r = 8$) è nettamente più efficace che aumentare il rango su pochi moduli. È emerso inoltre un trade-off critico sulla stabilità: LR elevati ($1e^{-3}$), pur funzionando su configurazioni semplici, causano *Training Collapse* quando applicati a modelli più complessi ($r \geq 16$ su tutti i layer).

Rango	Target Modules	LR	Acc	Esito
8	[q, v]	$1e^{-3}$	92.35%	Buon risultato base.
8	[q, v, k, dense]	$2e^{-4}$	94.20%	Migliore (Stabile).
16	[q, v, k, dense]	$1e^{-3}$	34.75%	<i>Training Collapse</i> (LR troppo alto).
32	[q, v]	$1e^{-4}$	91.75%	Aumentare r non aiuta quanto targettare dense .

Tabella 3: Selezione della configurazione LoRA ottimale tramite Grid Search

3.1.7 Sintesi Finale e Confronto

Il grafico seguente riassume il salto prestazionale ottenuto passando attraverso le diverse strategie.

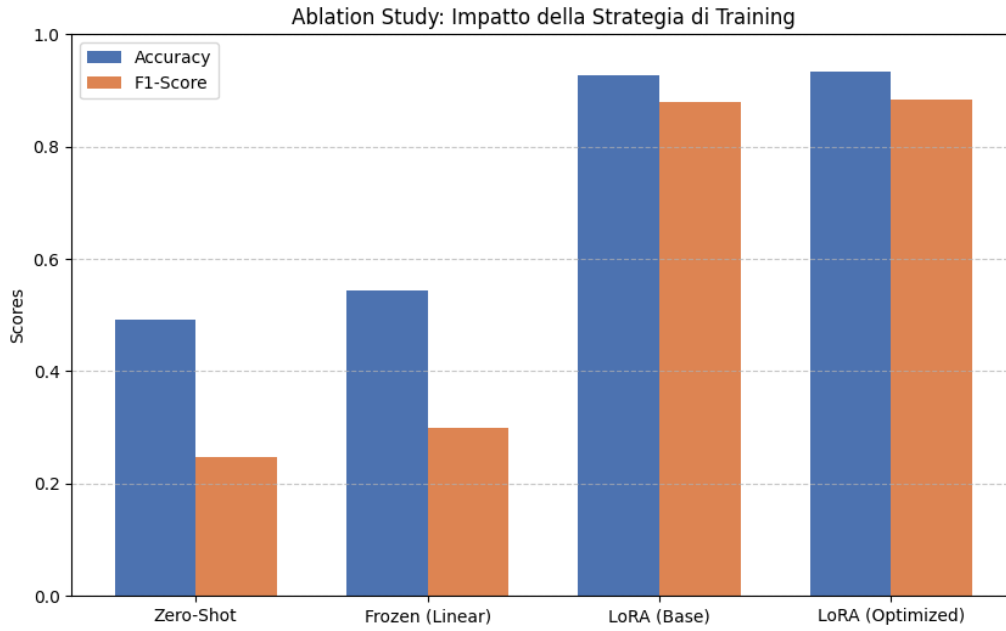


Figura 1: Confronto visivo delle Accuracy tra le strategie testate. Si nota il netto divario tra i metodi Frozen e LoRA.

In conclusione, la tabella riassuntiva delle migliori configurazioni per categoria è la seguente:

Strategia	Accuracy	F1	Conclusioni
In-Context ($k = 0$)	49.30%	24.6%	Inadatto per task emotivi specifici.
Transfer Learning	54.45%	29.9%	Limitato dalle feature pre-addestrate.
LoRA Base	92.35%	87.5%	Ottimo, ma migliorabile.
LoRA Optimized	94.20%	91.7%	Best Model. Estensione ai layer densi + LR controllato.

Tabella 4: Riepilogo finale Ablation Study

3.1.8 Results Discussion

I risultati ottenuti hanno ampiamente superato le aspettative iniziali per un approccio basato su PEFT. Aver raggiunto un'accuratezza del 94.20% dimostra che i modelli pre-addestrati su domini specifici (come BERTweet per i testi provenienti dai social media) costituiscono una base eccellente per il fine tuning, richiedendo l'aggiornamento di una frazione minima di parametri per adattarsi a task complessi. Inoltre, l'elevato Macro F1-Score (91.69%) è un indicatore cruciale: conferma che il modello non è collassato sulle classi maggioritarie (come *Joy* o *Sadness*), ma ha imparato a discriminare con efficacia anche le emozioni meno frequenti nel dataset.

Tuttavia, l'analisi qualitativa degli errori, supportata dalla Matrice di Confusione (Figura 2), ha rivelato che la maggior parte delle misclassificazioni residue non è casuale, ma si concentra su specifiche coppie di emozioni semanticamente vicine o contestualmente ambigue:

1. **Love vs Joy:** Questa rappresenta la confusione più frequente. È dovuta alla forte sovrapposizione semantica nei testi brevi tipici dei social network, dove parole di affetto, forte apprezzamento ed entusiasmo vengono usate in modo intercambiabile. Frasi che esprimono una profonda gioia spesso contengono termini legati all'amore (es. "I absolutely love this!"), rendendo il confine decisionale estremamente sfumato, a volte anche per un annotatore umano.
2. **Surprise vs Fear:** Nei tweet, che sono intrinsecamente brevi e spesso privi di un contesto esteso, la reazione a un evento scioccante può essere facilmente confusa dal modello: non avendo abbastanza indizi testuali, risulta difficile disambiguare se la sorpresa sia di natura neutra/positiva oppure legata a una minaccia (paura).

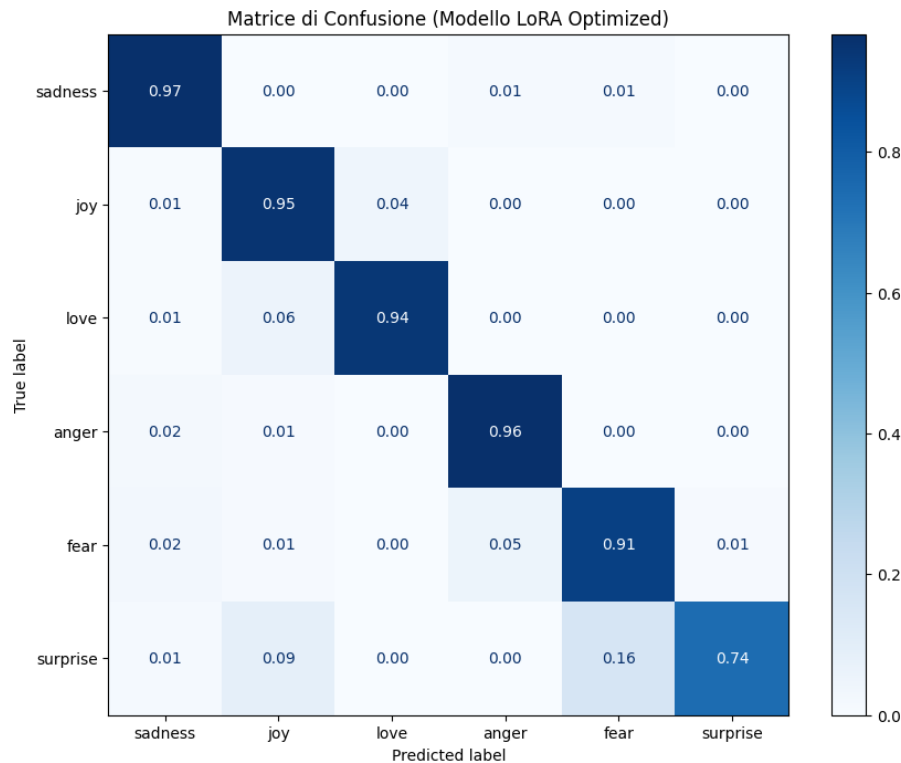


Figura 2: Matrice di Confusione del modello LoRA.

3.1.9 Method Validity

Il metodo LoRA si è dimostrato estremamente valido e robusto. Al contrario, lo studio di ablation su **In-Context Learning** ha evidenziato un limite strutturale critico: l'architettura BERTweet, progettata con una finestra di contesto ridotta (128 token), **non è idonea** a strategie Few-Shot ($k > 3$). Gli esempi dimostrativi tendono a saturare la memoria e troncare il prompt, peggiorando le performance invece di migliorarle, rendendo il fine-tuning una scelta obbligata per questo tipo di architetture.

3.2 distilbert-base-uncased

Il modello scelto è **DistilBERT-base-uncased**, un modello di tipo encoder derivato da BERT, progettato per essere più leggero e veloce mantenendo buone prestazioni.

Presenta le seguenti proprietà:

- Parametri: 66M
- Lingua: English
- Tensor type: F32
- Versione uncased
- Architettura Transformer encoder distillata

Il modello eredita il pre-training basato su:

- **Masked Language Modelling (MLM)**: viene mascherato casualmente il 15% delle parole nell'input; il modello deve prevedere i token mascherati a partire dal contesto circostante.

A differenza di BERT, DistilBERT non utilizza il task Next Sentence Prediction (NSP) durante il pre-training, rendendo il processo più efficiente e riducendo la complessità del modello.

3.2.1 Selezione dei parametri di training

Una fase cruciale in questo tipo di scenari è rappresentata da come viene svolto l'addestramento del modello. Questo è in larga parte delineato dalla selezione dei parametri di addestramento. Nel nostro caso ci siamo soffermati sui seguenti:

- Numero di epoche: quante volte l'intero training set viene processato
- Learning rate: la dimensione dei passi di addestramento
- Weight decay: evita che i parametri crescano troppo
- Numero di batch di training per device

Il resto dei parametri, come la politica di gestione del learning rate durante l'addestramento, sono stati lasciati di default.

3.2.2 Parametri per Classification Head Alignment

Learning Rate	3e-3
Weight Decay	0.01
Number of Epochs	15
Train Batch Size	16

Tabella 5: Parametri per classification head alignment

Per l'allineamento del classification head di Distilbert è stato utilizzato un learning rate pari a 3×10^{-3} , valore più elevato rispetto a quello tipicamente impiegato nel fine-tuning completo dei transformer. Questa scelta è coerente con lo scenario di light fine-tuning, in cui il backbone pre-addestrato rimane congelato e viene aggiornato il livello di classificazione, inizializzato casualmente. Un learning rate più alto consente quindi al classificatore

di adattarsi rapidamente alle rappresentazioni già strutturate fornite dal modello pre-addestrato, favorendo una convergenza più veloce senza compromettere la stabilità del training.

È stato inoltre impostato un weight decay pari a 0.01, con funzione di regolarizzazione L2 sui pesi del classification head. Questa scelta contribuisce a limitare la crescita eccessiva dei parametri, riducendo il rischio di overfitting e migliorando la capacità di generalizzazione.

Il batch size di 16 rappresenta un compromesso tra stabilità dell'ottimizzazione e vincoli computazionali, introducendo una moderata variabilità stocastica nei gradienti che può favorire la generalizzazione.

Il numero di epoche è stato fissato a 15: durante le prove è stato osservato che proseguendo oltre, anche per molte epoche aggiuntive, la validation loss rimaneva sostanzialmente stabile, con variazioni minime. Per questo motivo si è scelto di interrompere il training a 15 epoche, in quanto il modello aveva già raggiunto una fase di convergenza e ulteriori iterazioni non portavano miglioramenti significativi.

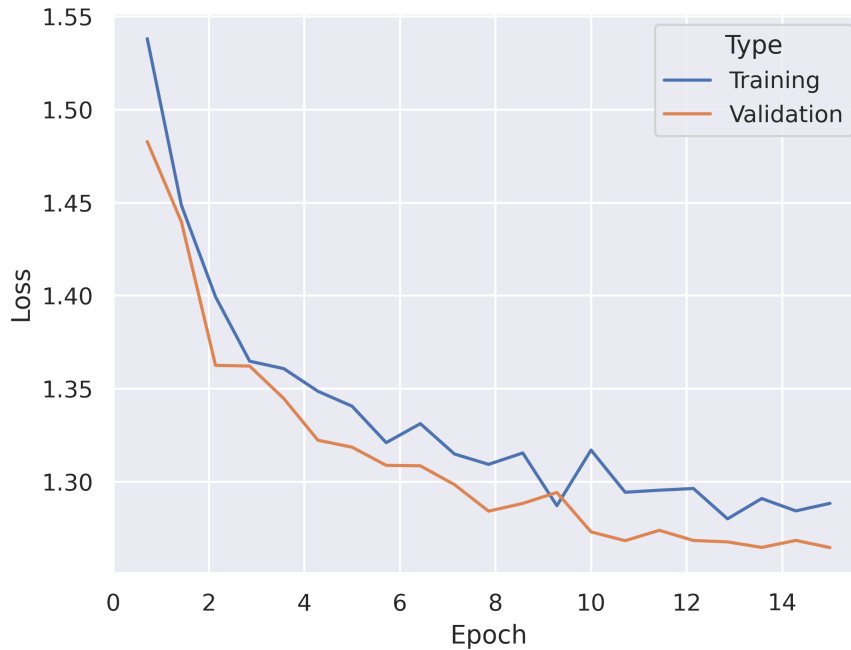


Figura 3: Training e Validation Loss - Classification Head Alignment

Il grafico della loss mostra un andamento decrescente sia per il training che per la validation, con una riduzione più marcata nelle prime epoche e un successivo rallentamento del miglioramento. Nella fase iniziale il classification head si adatta rapidamente alle rappresentazioni fornite dal backbone di DistilBERT, determinando un calo evidente della loss. Successivamente la diminuzione diventa più graduale, segno che il modello entra in una fase di raffinamento dei pesi.

Le curve di training e validation rimangono molto vicine lungo tutto il processo, senza divergenze significative. Questo comportamento indica assenza di overfitting marcato e una buona capacità di generalizzazione del classificatore. Nelle ultime epoche la validation loss tende a stabilizzarsi, mostrando variazioni molto contenute: tale plateau ha motivato

la scelta di interrompere il training a 15 epoche, poiché ulteriori iterazioni non avrebbero apportato miglioramenti sostanziali.

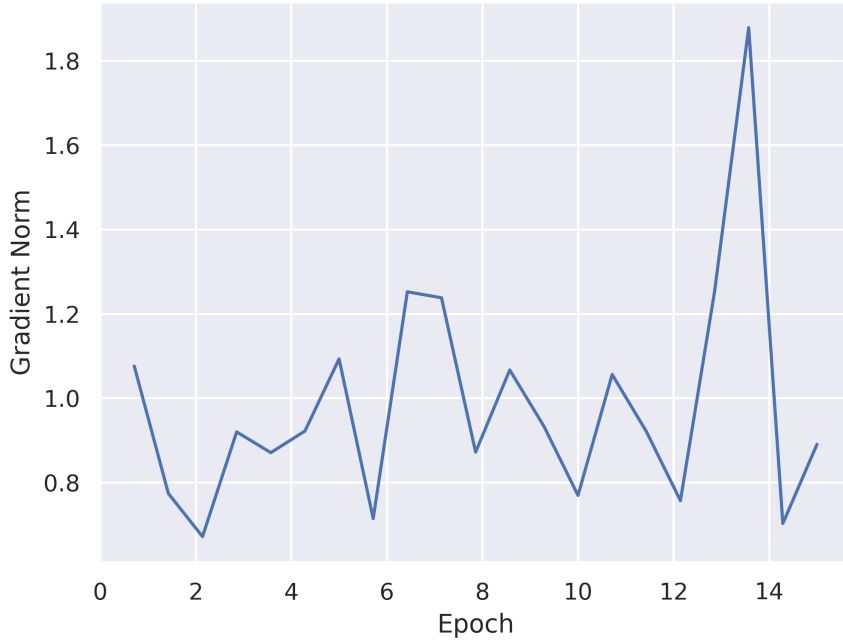


Figura 4: Gradient Norm - Classification Head Alignment

Il grafico della norma del gradiente evidenzia valori generalmente compresi in un intervallo stabile, con oscillazioni moderate tra un'epoca e l'altra. Questo comportamento è coerente con l'utilizzo di un batch size pari a 16, che introduce una componente stocastica negli aggiornamenti, e con il fatto che si stia ottimizzando principalmente il classification head. Le variazioni osservate riflettono la normale dinamica dell'ottimizzazione in un contesto non deterministico.

Si nota un picco isolato nelle ultime epoche, ma non si osservano fenomeni di esplosione persistente del gradiente né instabilità prolungate. Il valore rientra rapidamente nei range precedenti, indicando che il processo di training rimane numericamente stabile. Nel complesso, l'andamento della norma del gradiente conferma che l'ottimizzazione è avvenuta in modo controllato e coerente con un corretto allineamento del classification head alle rappresentazioni interne di DistilBERT.

3.2.3 Parametri Fine-tuning Completo

Learning Rate	3e-5
Weight Decay	0.01
Max Epochs	14
Batch Size	16

Tabella 6: Parametri per fine-tuning completo

Per il fine-tuning completo del modello DistilBERT, sono stati adottati parametri più conservativi rispetto alla fase di addestramento della sola classification head, in quanto in questa fase vengono aggiornati tutti i pesi del modello pre-addestrato.

Il learning rate pari a 3×10^{-5} è stato scelto in linea con le pratiche consolidate per il fine-tuning dei modelli Transformer pre-addestrati. In letteratura e nella documentazione ufficiale dei modelli basati su BERT e DistilBERT, il range tipicamente raccomandato per il learning rate è compreso tra $1e-5$ e $5e-5$. L'utilizzo di un valore contenuto consente di preservare le conoscenze linguistiche acquisite durante il pre-training, evitando aggiornamenti troppo aggressivi che potrebbero degradare le rappresentazioni interne del modello.

Il weight decay pari a 0.01 è stato mantenuto per introdurre regolarizzazione L2 e migliorare la capacità di generalizzazione, limitando fenomeni di overfitting durante l'aggiornamento di tutti i parametri del modello.

Il numero massimo di 14 epoche è stato inizialmente impostato considerando che, nel fine-tuning di modelli pre-addestrati, sono generalmente sufficienti poche epoche per adattare efficacemente il modello al task specifico, mentre un numero eccessivo potrebbe aumentare il rischio di overfitting. Tuttavia, dall'analisi dell'andamento della loss si è osservato che i valori migliori di validation loss si raggiungono già intorno all'epoca 1.75. Dopo questo punto, la training loss continua a diminuire rapidamente, mentre la validation loss inizia progressivamente ad aumentare, evidenziando un chiaro fenomeno di overfitting. Per questo motivo, pur avendo impostato 14 epoche come limite massimo, si è scelto di interrompere il training anticipatamente all'epoca 1.75, corrispondente alla configurazione con le migliori prestazioni in termini di generalizzazione nel fine-tuning completo.

Infine, la batch size pari a 16 rappresenta un compromesso tra stabilità dell'aggiornamento del gradiente e limiti di memoria GPU, garantendo un training efficiente e stabile.

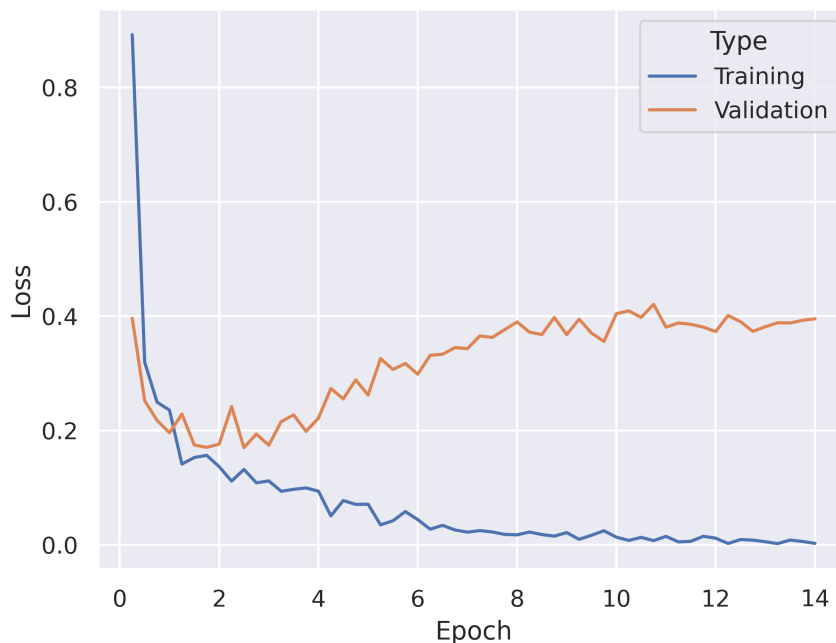


Figura 5: Training e Validation Loss - Fine-tuning Completo

Nel grafico viene riportato l'andamento della training loss e della validation loss nel corso delle epoche.

Si osserva che:

1. La training loss decresce rapidamente fin dalle prime epoche, passando da un valore iniziale elevato (circa 0.9) a valori prossimi allo zero nelle epoche successive.
2. La validation loss, invece, diminuisce solo nelle primissime fasi del training, raggiungendo il valore minimo intorno all'epoca 1.75.

Dopo l'epoca 1.75, mentre la training loss continua a diminuire, la validation loss inizia progressivamente ad aumentare. Questo andamento evidenzia chiaramente un fenomeno di **overfitting**: il modello continua a migliorare sulle istanze di training, ma peggiora sulle istanze di validazione, perdendo capacità di generalizzazione.

L'analisi del grafico conferma quindi la scelta metodologica di applicare *early stopping* all'epoca 1.75, nonostante il limite massimo fosse stato impostato a 14 epoche. Il punto di minimo della validation loss rappresenta infatti la configurazione con il miglior compromesso tra apprendimento del task e mantenimento della capacità di generalizzazione.

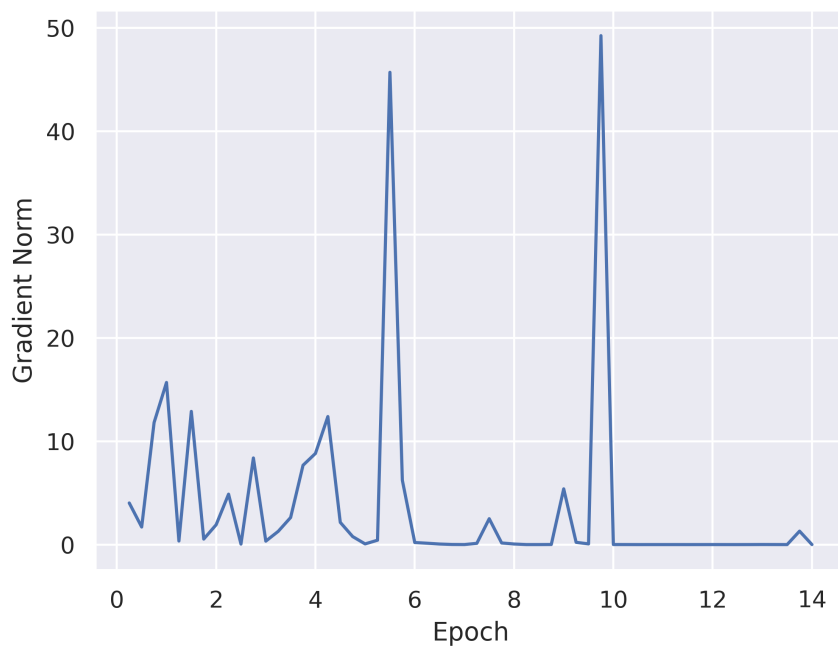


Figura 6: Gradient Norm - Fine-tuning Completo

L'andamento della Gradient Norm mostra valori inizialmente più elevati e variabili nelle prime epoche, segno che il modello sta effettuando aggiornamenti significativi per adattarsi al nuovo task.

Intorno all'epoca 1.75 il modello ha già compiuto la parte più rilevante dell'apprendimento. Successivamente, la norma del gradiente tende a ridursi e stabilizzarsi su valori molto bassi, indicando una progressiva convergenza sul training set.

Poiché dopo l'epoca 1.75 la validation loss inizia ad aumentare, la diminuzione della gradient norm non corrisponde più a un miglioramento della generalizzazione ma a un adattamento eccessivo ai dati di training.

Pertanto, l'andamento della gradient norm risulta coerente con la scelta di applicare l'early stopping all'epoca 1.75, punto in cui il modello raggiunge il miglior compromesso tra apprendimento e capacità di generalizzazione.

3.2.4 Risultati

Di seguito sono riportate le performance dei migliori modelli, light fine-tuned e complete fine-tuned.

Performance inContext Learning

In questa sezione si valuta quanto il modello, allineato al task, riesca a performare utilizzando la tecnica di *in-context learning* applicata al test set.

La valutazione è stata eseguita mediante tre diversi gradi di somministrazione:

1. **One Shot:** i sample sono arricchiti dalla presenza di un esempio di applicazione del task.
2. **Three Shot:** i sample sono arricchiti dalla presenza di tre esempi di applicazione del task.
3. **Five Shot:** i sample sono arricchiti dalla presenza di cinque esempi di applicazione del task.

One-Shot Performance Accuracy: 0.4805

Class	Precision	Recall	F1
0	0.4575	0.5835	0.5129
1	0.5577	0.7367	0.6348
2	0.0000	0.0000	0.0000
3	0.0000	0.0000	0.0000
4	0.3235	0.4911	0.3901
5	0.0000	0.0000	0.0000

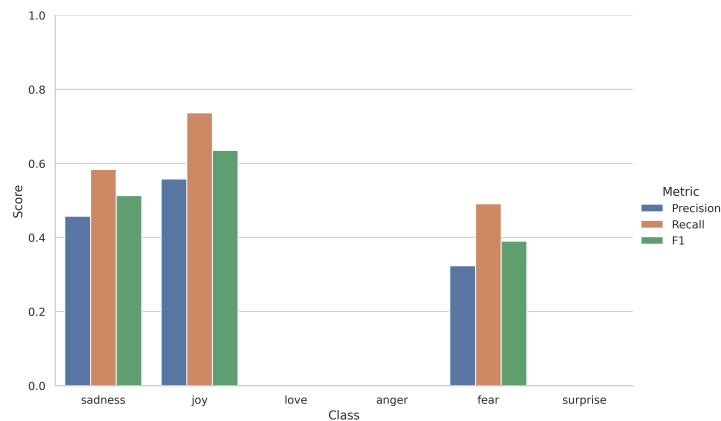


Figura 7: Metriche One Shot

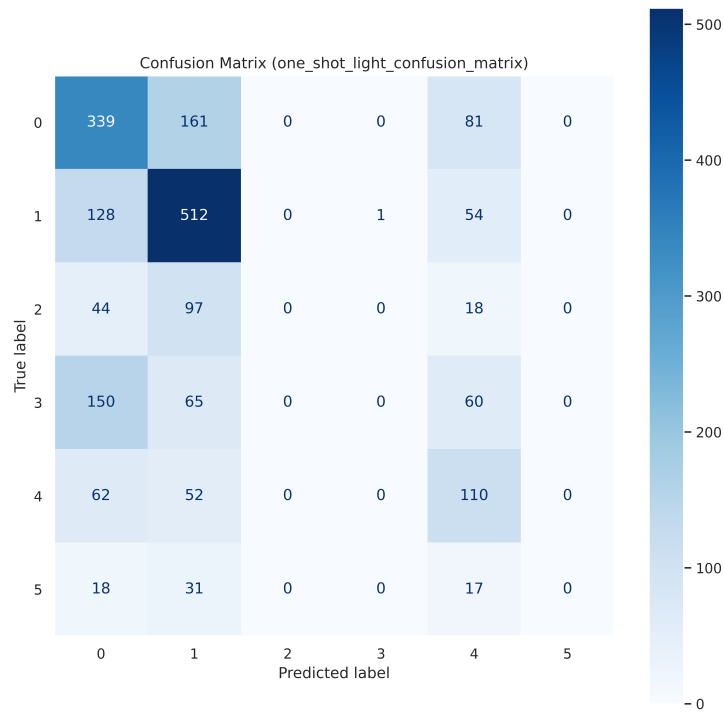


Figura 8: Confusion Matrix One Shot

Three-Shot Performance Accuracy: 0.4540

Class	Precision	Recall	F1
0	0.5103	0.3838	0.4381
1	0.4350	0.9525	0.5972
2	0.0000	0.0000	0.0000
3	0.0000	0.0000	0.0000
4	0.5610	0.1027	0.1736
5	0.0000	0.0000	0.0000

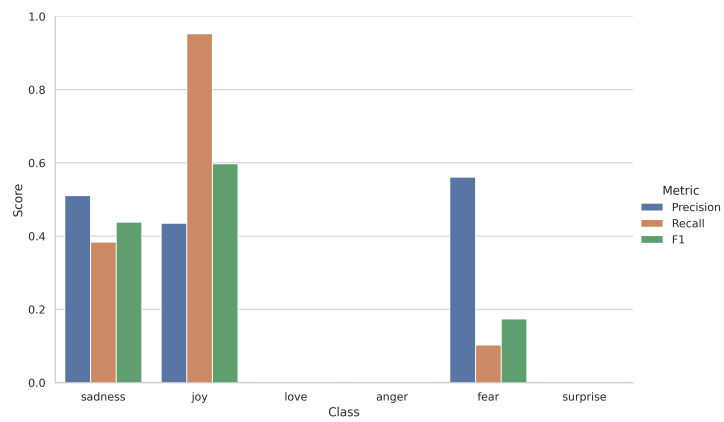


Figura 9: Metriche Three Shot

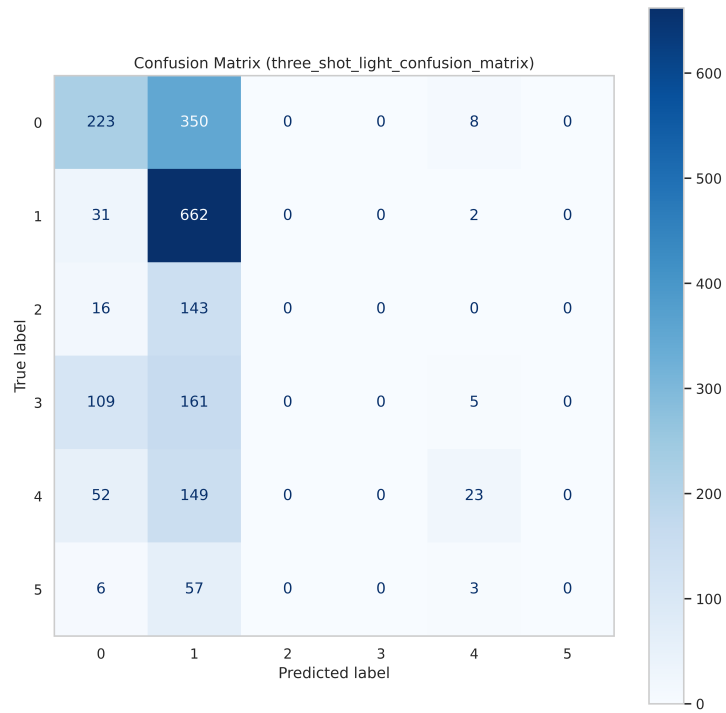


Figura 10: Confusion Matrix Three Shot

Five-Shot Performance Accuracy: 0.4730

Class	Precision	Recall	F1
0	0.4152	0.7917	0.5447
1	0.6433	0.5554	0.5961
2	0.0000	0.0000	0.0000
3	0.0000	0.0000	0.0000
4	0.3425	0.4464	0.3876
5	0.0000	0.0000	0.0000

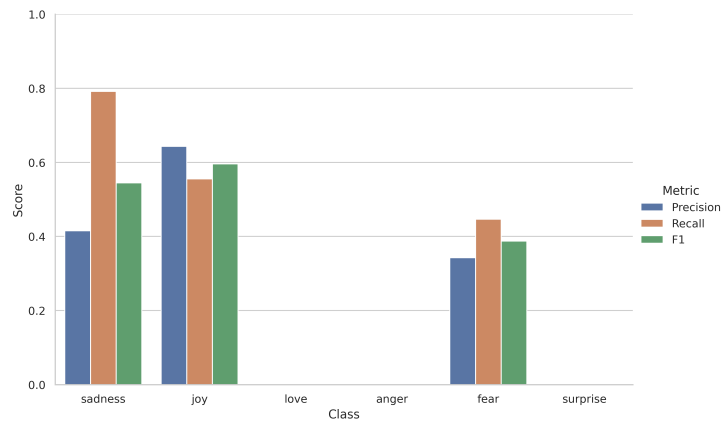


Figura 11: Metriche Five Shot

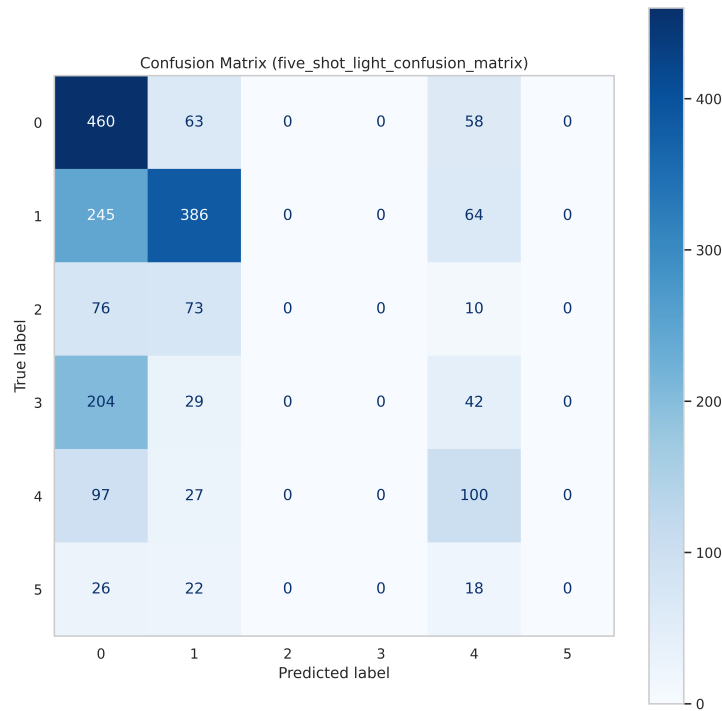


Figura 12: Confusion Matrix Five Shot

Confronto Performance In-Context In questa sezione si confrontano le performance dei vari gradi di inContext, ma per rendere il confronto più comprensibile, si è scelto di restringere le metriche di valutazione a F1 e Accuracy.

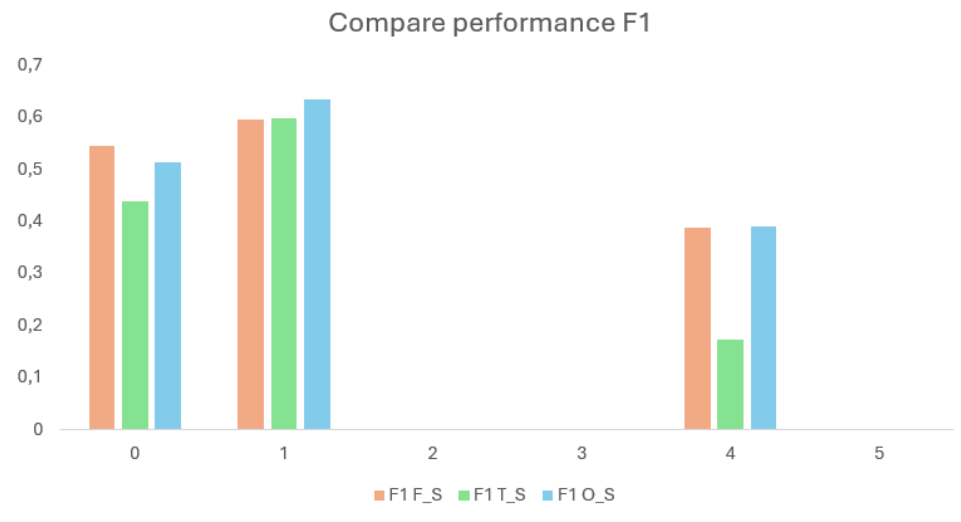


Figura 13: Confronto F1-score

Il grafico confronta l'F1-score ottenuto nelle diverse configurazioni sperimentali (Five Shot, Three Shot e One Shot) per ciascuna classe. Si osserva che la classe 1 mantiene le prestazioni migliori in tutte le configurazioni, con valori di F1 prossimi a 0.6, confermandosi la categoria più facilmente discriminabile dal modello.

La classe 0 mostra prestazioni intermedie, con un miglioramento nella configurazione Five Shot rispetto alle altre. La classe 4, invece, presenta valori sensibilmente più bas-

si, evidenziando una maggiore difficoltà del modello nel riconoscere correttamente tale categoria.

Nel complesso, le prestazioni risultano sbilanciate tra le classi: alcune categorie vengono apprese in modo più efficace, mentre altre rimangono più problematiche.

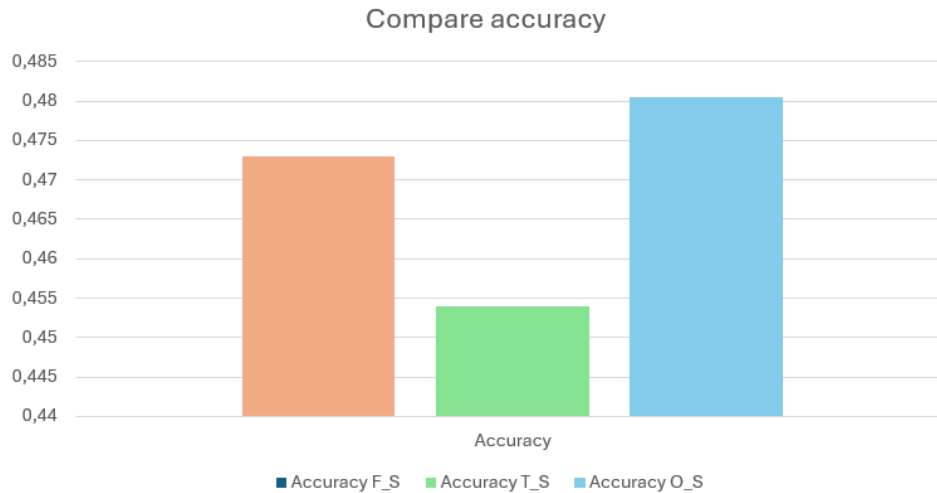


Figura 14: Confronto Accuracy

Il grafico evidenzia che l'accuracy globale non cresce in modo monotono all'aumentare degli shot: si osserva una diminuzione nel three-shot rispetto all'one-shot, seguita da un miglioramento nel five-shot. Tuttavia, i valori rimangono contenuti, coerentemente con la forte polarizzazione del modello verso una singola classe. L'accuracy risulta quindi influenzata dallo sbilanciamento delle predizioni, non riflettendo un apprendimento uniforme su tutte le classi.

3.2.5 Performance Fine-tuning Completo

In questa sezione si espongono le performance del modello sul test set senza l'applicazione della tecnica di inContext.

Accuracy: 0.9260

Class	Precision	Recall	F1
0	0.9621	0.9604	0.9612
1	0.9406	0.9568	0.9486
2	0.8553	0.8176	0.8360
3	0.9211	0.9211	0.9211
4	0.8692	0.9196	0.8937
5	0.8163	0.6061	0.6957

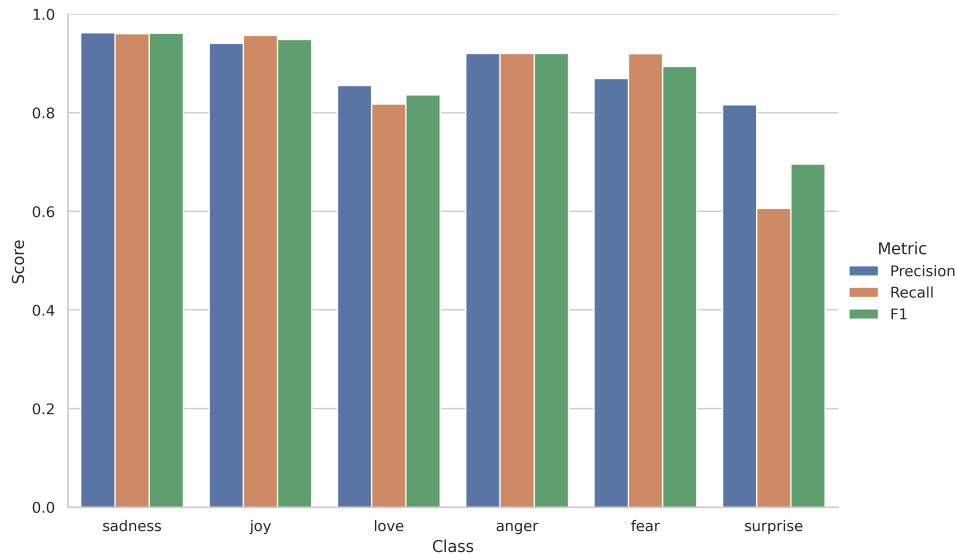


Figura 15: Metriche per classe - Fine-tuning Completo

Il grafico mostra Precision, Recall e F1-score per ciascuna classe emotiva dopo il fine-tuning completo del modello. Le prestazioni risultano complessivamente elevate e bilanciate per la maggior parte delle classi. In particolare, sadness, joy e anger presentano valori prossimi a 0.95–0.97, indicando un’elevata capacità del modello di identificare correttamente tali categorie con un buon equilibrio tra precisione e richiamo.

Anche fear e love mostrano risultati solidi, sebbene con una lieve variabilità tra precision e recall. La classe surprise risulta invece la più complessa da classificare, con una riduzione più marcata soprattutto nel recall, suggerendo una maggiore difficoltà del modello nel riconoscere tutti i casi appartenenti a questa categoria.

Nel complesso, il fine-tuning completo consente al modello di apprendere rappresentazioni discriminative efficaci, garantendo performance elevate e relativamente omogenee tra le diverse classi, con limitate criticità concentrate principalmente sulla classe surprise.

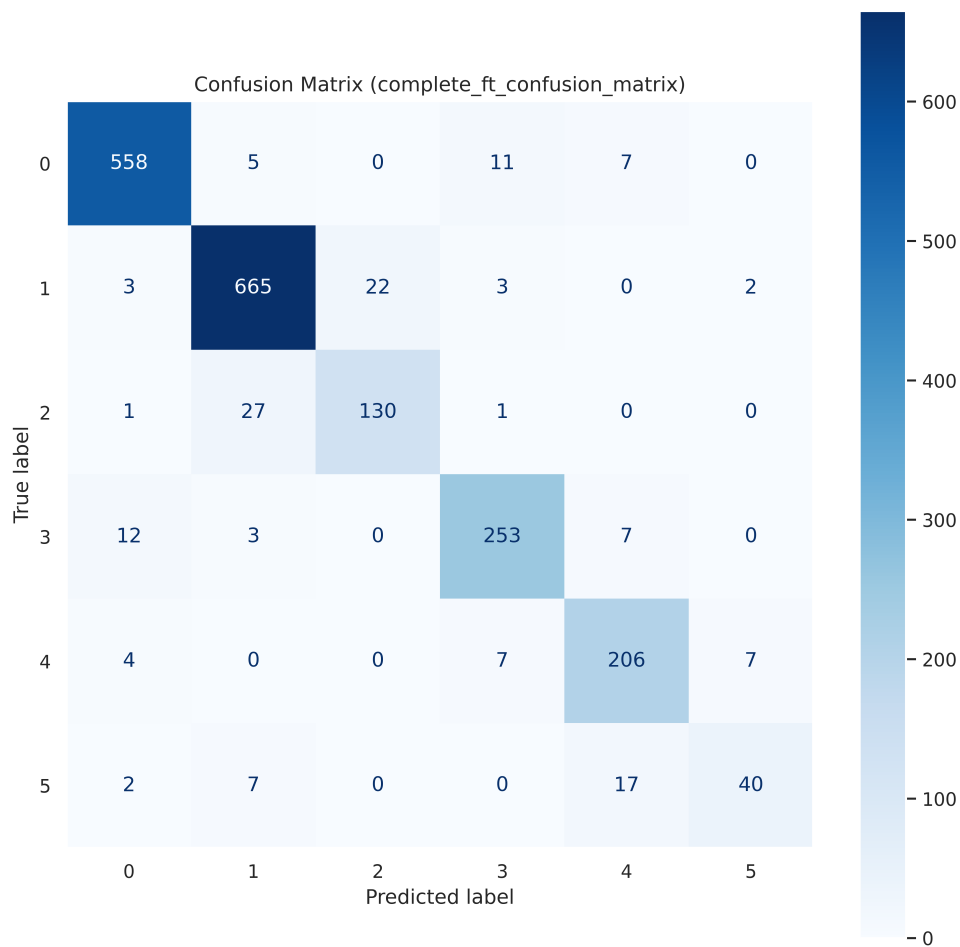


Figura 16: Confusion Matrix - Fine-tuning Completo

La matrice di confusione evidenzia le prestazioni del modello nella classificazione delle 6 classi. I valori più elevati si trovano lungo la diagonale principale, indicando un buon numero di predizioni corrette per tutte le classi.

In particolare, la classe 2 mostra la performance migliore con 665 classificazioni corrette, seguita dalla classe 1 con 558 predizioni corrette. Anche le classi 3, 4 e 5 presentano buoni risultati, rispettivamente con 130, 253 e 206 casi corretti, e un livello di errore complessivamente contenuto.

La classe 6 risulta la più critica: pur avendo 40 classificazioni corrette, mostra una confusione significativa soprattutto con la classe 5 (17 casi), oltre a qualche errore verso le classi 1 e 2. Questo suggerisce una possibile somiglianza tra le classi 5 e 6 o una maggiore difficoltà del modello nel distinguere correttamente questa categoria.

Nel complesso, il modello dimostra buone prestazioni generali, con errori limitati e concentrati principalmente tra specifiche coppie di classi.

3.3 bert-base-uncased

Il modello scelto è **Bert-base-uncased**, un modello di tipo encoder con le seguenti proprietà:

- Parametri: 110M

- Lingua: English
- Tensor type: F32
- Uncased: English = english
- [CLS] vector

Il modello ha subito una fase di pre-training suddivisa in due momenti definiti dai task somministrati:

- **Masked Language Modelling** : vengono mascherate casualmente il 15% delle parole nell'input; quindi, si esegue l'intera frase mascherata attraverso il modello che deve prevedere le parole mascherate.
- **Next Sentence Predictor** : vengono concatenate due frasi mascherate come input. A volte corrispondono a frasi contigue nel testo originale, a volte no. Il modello deve quindi prevedere se le due frasi si susseguono o meno.

3.3.1 Selezione dei parametri di training

Una fase cruciale in questo tipo di scenari è rappresentata da come viene svolto l'addestramento del modello. Questo è in larga parte delineato dalla selezione dei parametri di addestramento. Nel nostro caso ci siamo soffermati sui seguenti:

- Numero di epoche: quante volte l'intero training set viene processato
- Learning rate: la dimensione dei passi di addestramento
- Weight decay: evita che i parametri crescano troppo
- Numero di batch di training per device
- Floating point: precisione di rappresentazione dei tensori

Il resto dei parametri, come la politica di gestione del learning rate durante l'addestramento, sono stati lasciati di default.

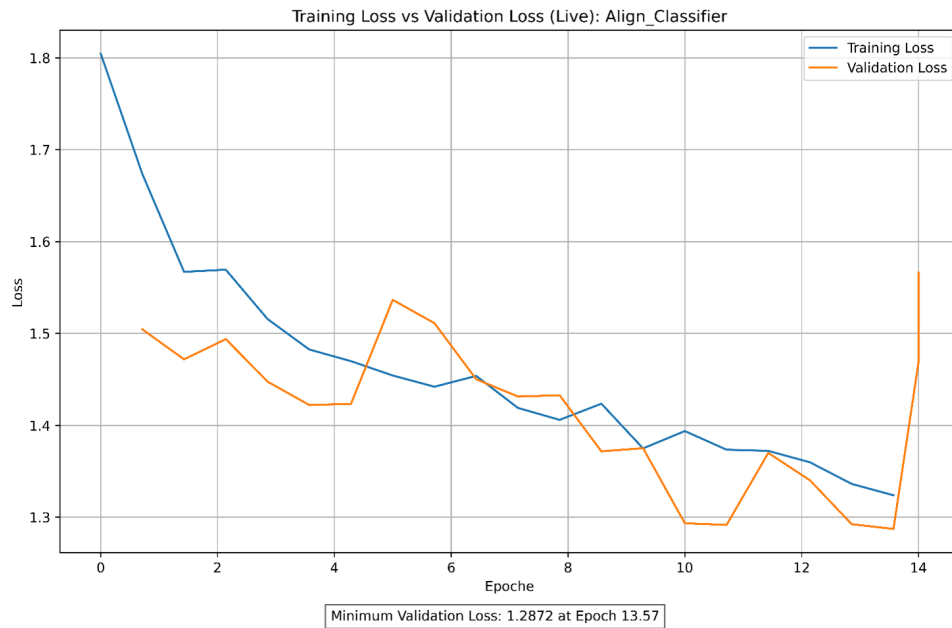
3.3.2 Parametri per Classification Head Alignment

Learning Rate	4e-3
Weight Decay	0.01
Number of Epochs	14
Train Batch Size	16
Fp16	True

Tabella 7: Parametri per classification head alignment

La scelta di questi parametri è motivata da diversi fattori. In primo luogo, si possono eseguire passi di addestramento più ampi, questo perché il livello in esame dev'essere addestrato da zero; quindi, non c'è il pericolo di inquinare i risultati di pre-training.

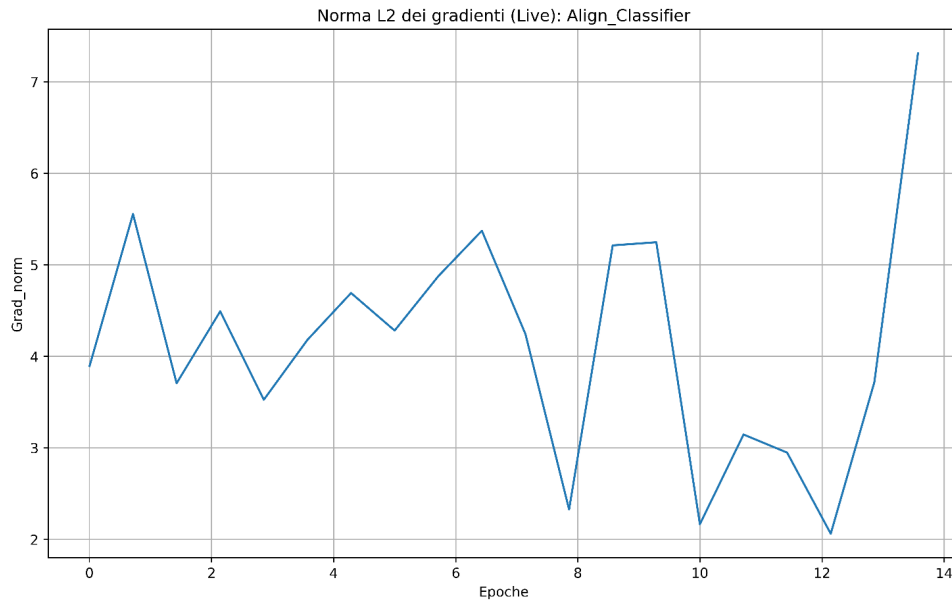
Inoltre, per la decadenza dei pesi si è tenuto un valore abbastanza standar, in quanto si è osservato che la rumorosità introdotta da esso, accentuata dalla limitata dimensione dei batch, non riscontri grande impatto sull'apprendimento.



Osservando l'andamento della training loss, possiamo affermare che, si delinea una tendenza discendente, la quale suggerisce un apprendimento sano. Infatti, non si denota alcun asintoto orizzontale che possano far sospettare la presenza di un minimo locale, che l'addestramento non riesce a superare.

L'addestramento prosegue sino alla soglia delle 14 epoche, questo perché, osservando l'andamento della validation loss, si può concludere che il modello migliora sino alla 13.57 epoca, oltre la quale la validation loss riscontra una crescita non trascurabile, il che può essere sintomo di over-fitting. Al termine del training viene prelevato il modello che ottiene un miglior risultato in fase di validation.

Si ricorda, che questa fase comprende solo l'ultimo livello del modello, per cui si ritiene difficile ottenere risultati migliori.



Per concludere l'analisi della fase di allineamento, si riporta anche l'andamento della norma L2 dei gradienti. Osservare questo parametro è essenziale per riconoscere fenomeni come il gradient vanishing e/o exploding. Nel nostro caso, la norma è stabile oscillando tra i valori 2 e 7.5, questo ci fa concludere che l'addestramento è stabile.

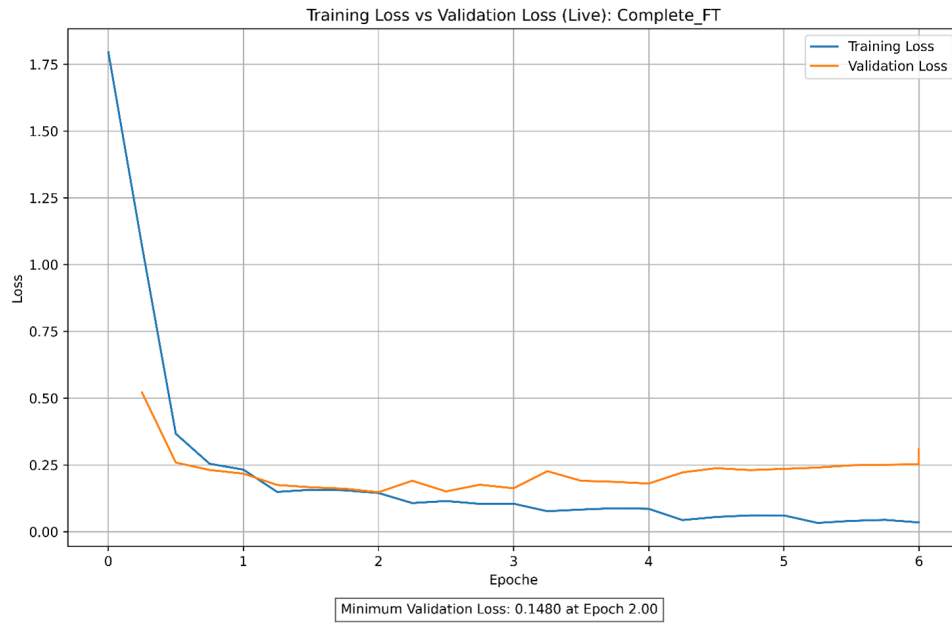
3.3.3 Parametri Fine-tuning Completo

Learning Rate	2e-5
Weight Decay	0.001
Max Epochs	6
Batch Size	16
Fp16	True

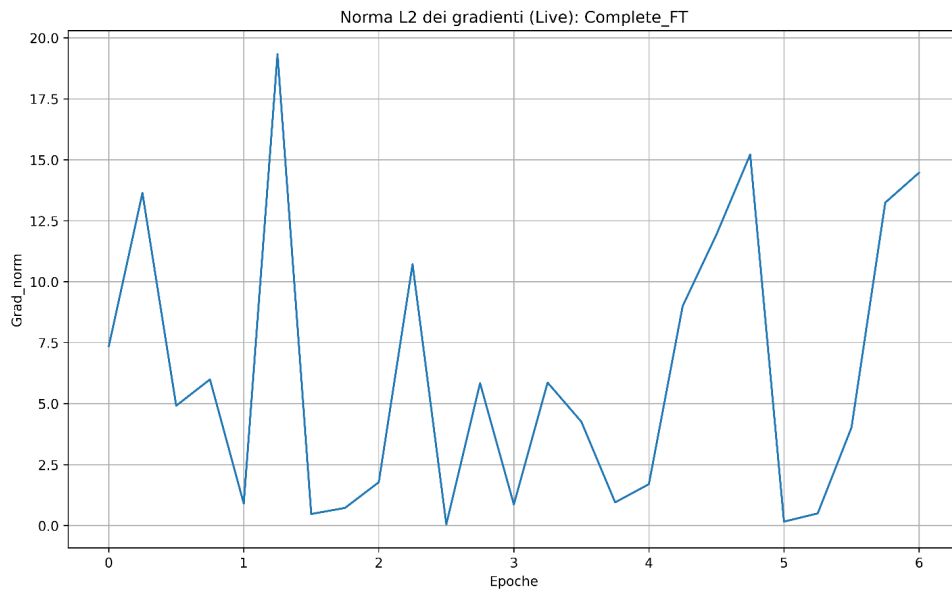
Tabella 8: Parametri per fine-tuning completo

In questa fase di addestramento siamo stati più cauti, spostandoci di meno e con una minore decadenza dei pesi, il che causa meno rumore.

Questo riportato di seguito è un addestramento standard, si poteva eseguirne un altro più completo. Una volta scelti i parametri di training, si poteva proseguire utilizzando il devset (training + validation) con una suddivisione casuale tra training set e validation set, ripetere più volte l'addestramento e calcolare media e varianza della training loss nel punto in cui la validation loss raggiungeva il suo minimo. Alla fine si eseguiva un unico training su tutto il devset fermando l'addestramento quando la training loss raggiunge la media delle training loss precedenti.



Si ripetono qui le stesse osservazioni fatte per la fase di allineamento. Tuttavia, a differenza della precedente fase, qui non si riscontra un'improvvisa risalita della validation loss, ma è più graduale; quindi, abbiamo proseguito l'addestramento per più tempo, così da essere ragionevolmente sicuri che questa non riprendesse più a scendere.



Per quanto riguarda lo studio della norma dei gradienti, in questo caso c'è una maggiore oscillazione, l'intervallo dei valori è maggiore. Tuttavia, si resta su valori sicuri e, inoltre, confrontandola con la training loss, si può riscontrare che gli spike isolati non hanno ripercussioni sull'addestramento.

3.3.4 Risultati

Di seguito sono riportate le performance dei migliori modelli, light fine-tuned e complete fine-tuned.

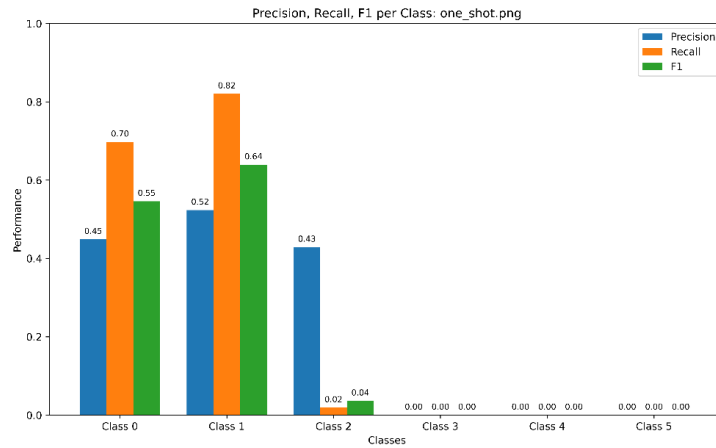
Performance inContext Learning

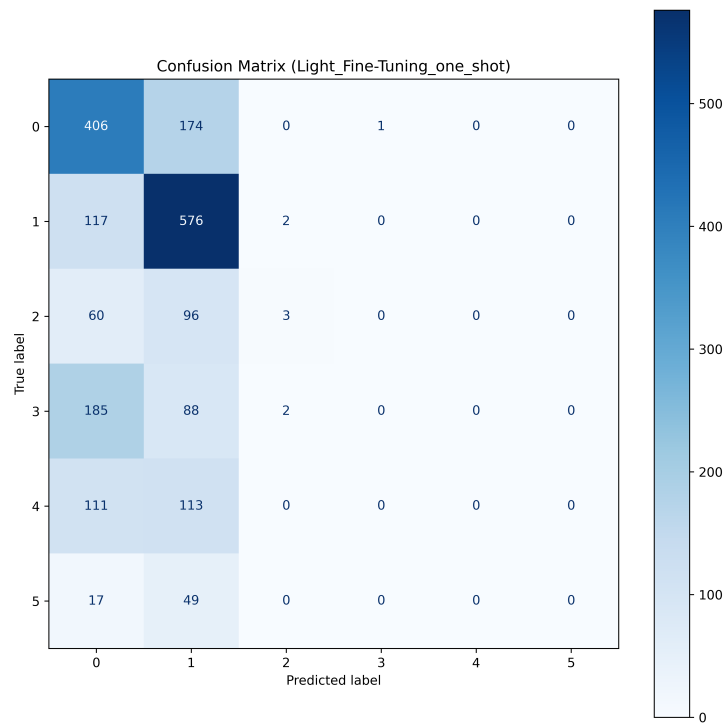
Qui si valuta quanto il modello, allineato al task, riesca a performare utilizzando la tecnica di inContext learning applicata al test set. La valutazione è stata eseguita mediante 3 gradi di somministrazione:

1. **One Shot:** i sample sono arricchiti dalla presenza di un esempio di applicazione del task.
2. **Three Shot:** i sample sono arricchiti dalla presenza di tre esempi di applicazione del task.
3. **Five Shot:** i sample sono arricchiti dalla presenza di cinque esempi di applicazione del task.

One-Shot performance Accuracy: 0.489

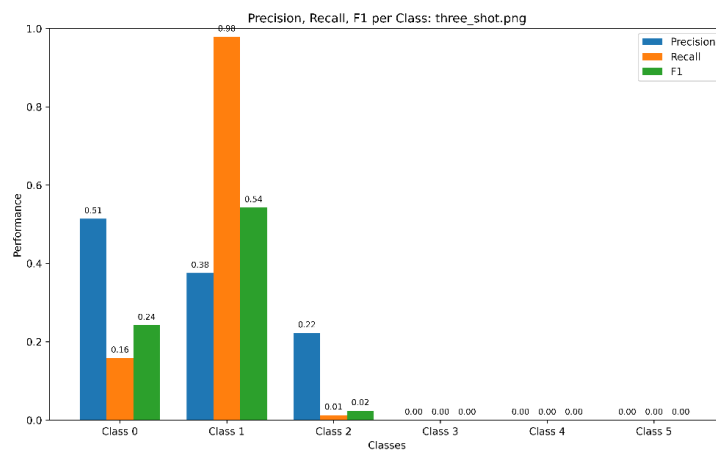
Class	Precision	Recall	F1
0	0.4485	0.6970	0.5458
1	0.5234	0.8201	0.6390
2	0.4285	0.0189	0.0361
3	0.0000	0.0000	0.0000
4	0.0000	0.0000	0.0000
5	0.0000	0.0000	0.0000

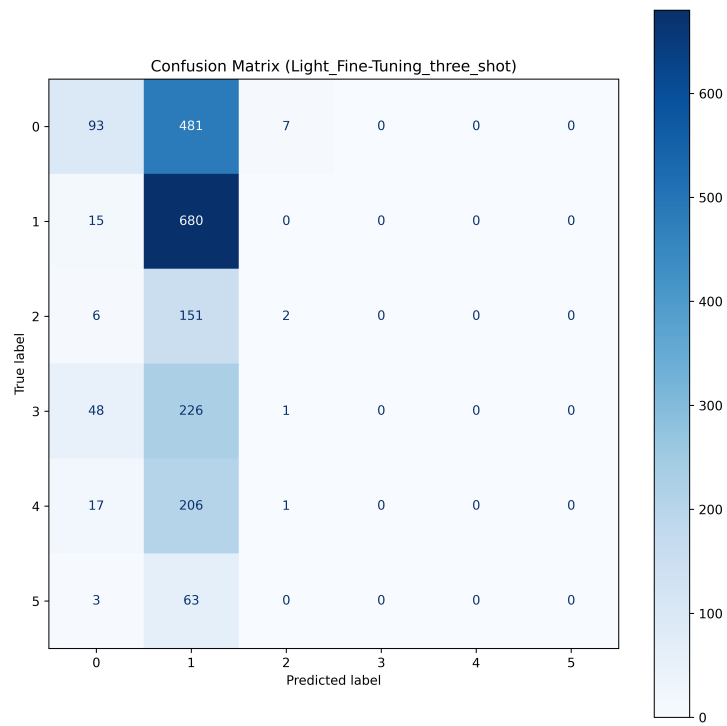




Three-Shot Accuracy: 0.387

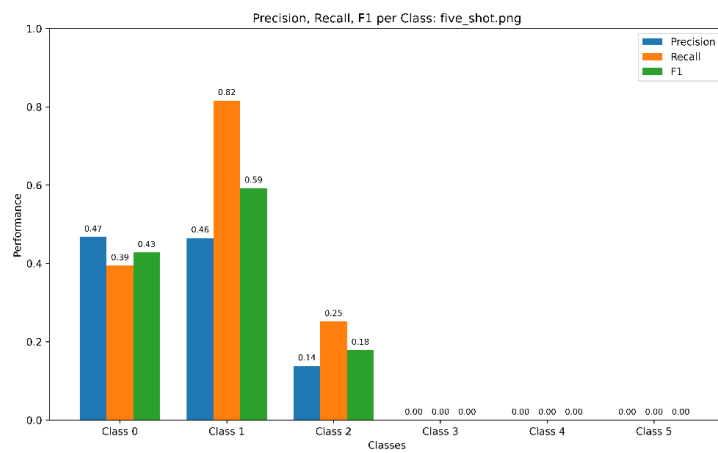
Class	Precision	Recall	F1
0	0.5140	0.1583	0.2421
1	0.3752	0.9784	0.5425
2	0.2222	0.0126	0.0238
3	0.0000	0.0000	0.0000
4	0.0000	0.0000	0.0000
5	0.0000	0.0000	0.0000

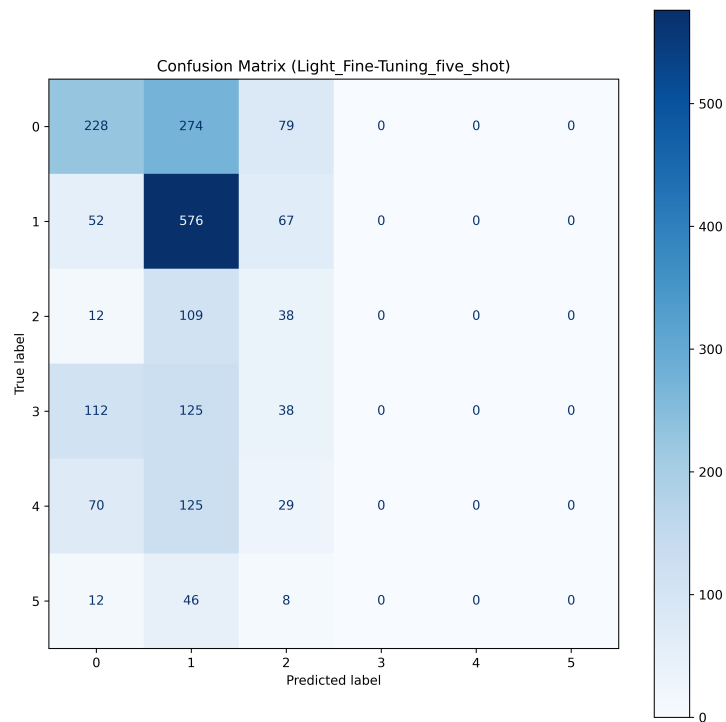




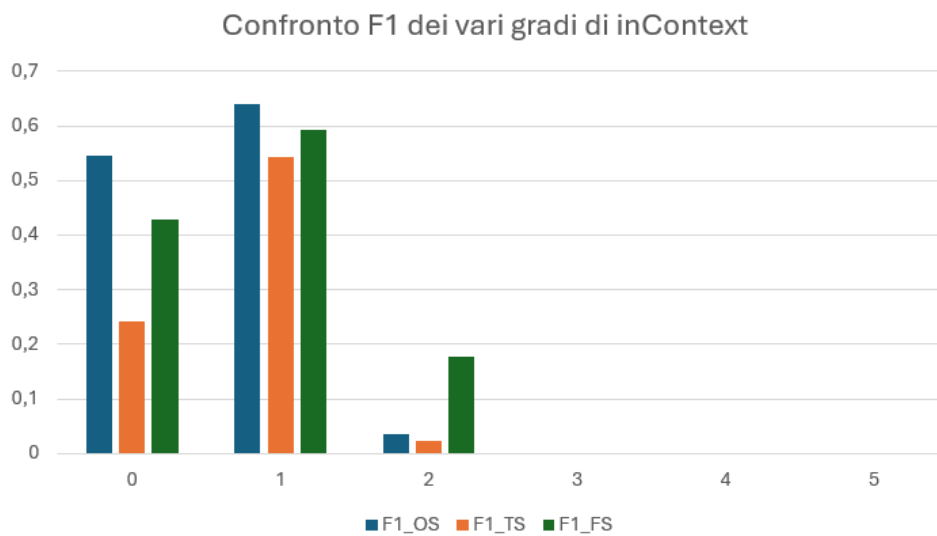
Five-Shot Accuracy: 0.418

Class	Precision	Recall	F1
0	0.4673	0.3941	0.4276
1	0.4647	0.8158	0.5922
2	0.1379	0.2516	0.1782
3	0.0000	0.0000	0.0000
4	0.0000	0.0000	0.0000
5	0.0000	0.0000	0.0000





Confronto Performance In-Context In questa sezione si confrontano le performance dei vari gradi di inContext.



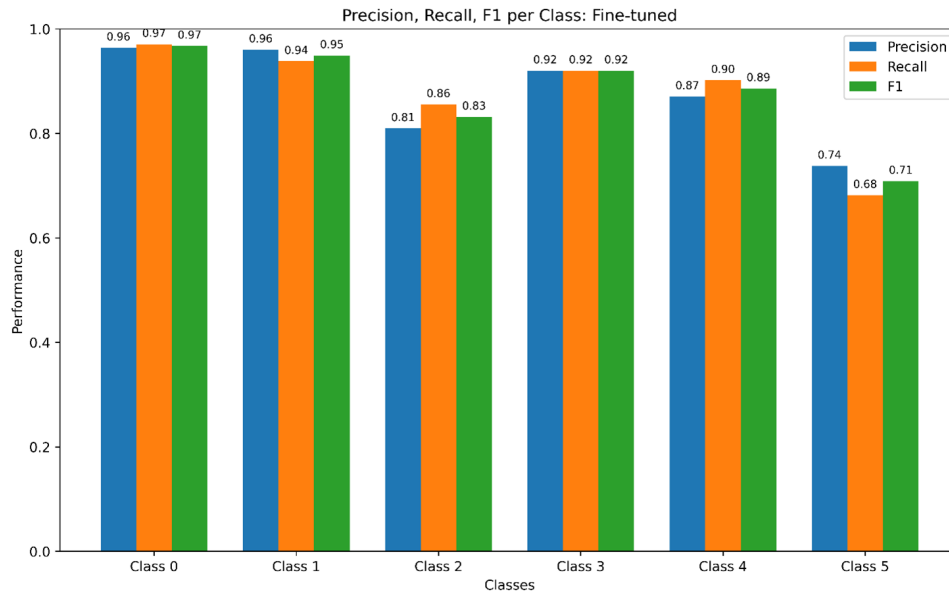
Osservando le metriche a confronto, si può riscontrare l'assenza di nette variazioni nelle prestazioni. Incrementando il numero di esempi in input per il modello, questo non incrementa le sue capacità. Questo conferma la natura di abilità emergente, dei modelli decoder di grandi dimensioni, dell'abilità esercitata.

3.3.5 Performance Fine-tuning Completo

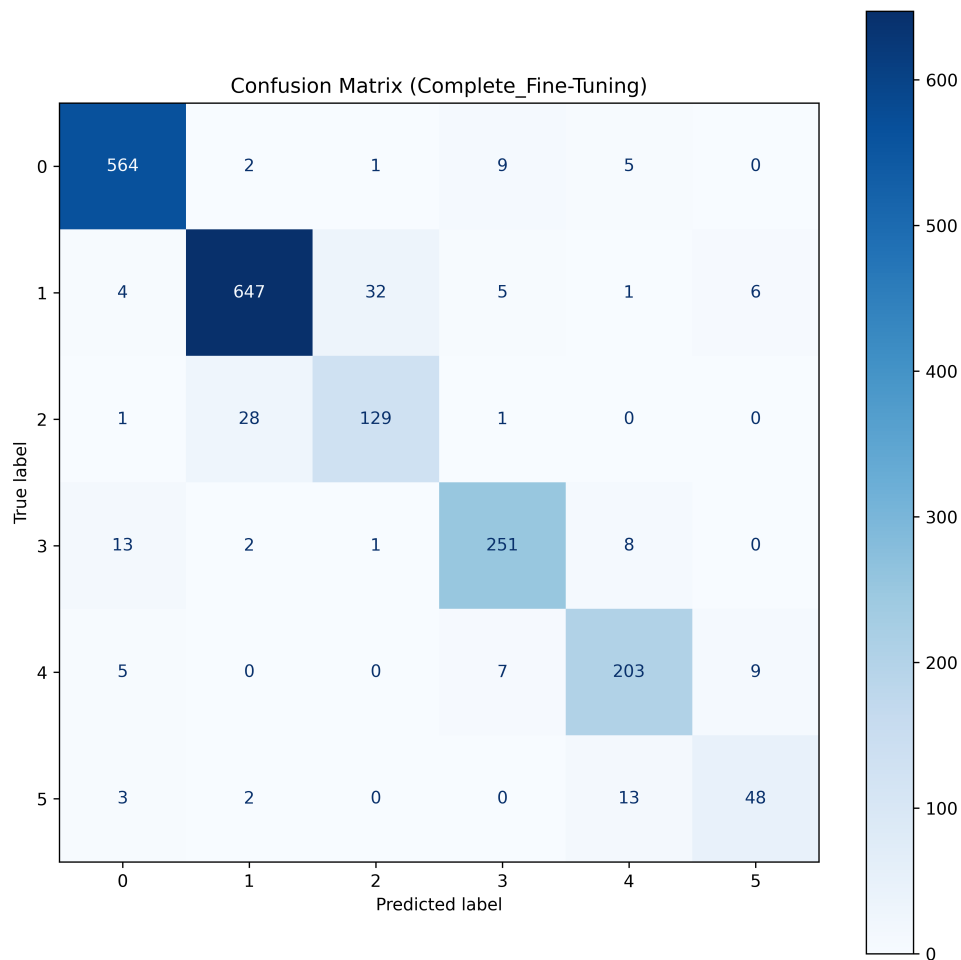
In questa sezione si espongono le performance del modello sul test set senza l'applicazione della tecnica di inContext.

Accuracy: 0.93

Class	Precision	Recall	F1
0	0.9641	0.9707	0.9674
1	0.9602	0.9381	0.9490
2	0.8095	0.8553	0.8318
3	0.9200	0.9200	0.9200
4	0.8707	0.9018	0.8860
5	0.7377	0.6818	0.7087



Si osserva un netto incremento delle prestazioni del modello. Tuttavia, si sottolinea che queste sono direttamente proporzionali alle dimensioni delle partizioni dei datasets. Il modello ottiene risultati migliori in presenza delle classi predominanti nei dati (Classe 0 e 1), al contrario, classi più rare (Classe 2 e 5) mettono in difficoltà il modello. In conclusione, le prestazioni del modello riflettono la natura sbilanciata dei datasets.



3.4 roberta-base

3.4.1 Model

RoBERTa sta per **Robustly optimized BERT** approach ed è un modello pre-addestrato con l'obiettivo di Masked Language Modeling (MLM). Data in input una frase, il modello maschera casualmente il 15% delle parole, elaborando l'intera frase mascherata e deve predire le parole che sono state oscurate. Il modello in questione (RoBERTa-base) si presenta con un numero di parametri allenabili pari a 124,650,246.

3.4.2 Demonstration and Technologies

L'analisi del modello è stata eseguita utilizzando le seguenti tecnologie:

- **Environment:** Google Colab (Python 3) con runtime T4 GPU.
- **Librerie:**
 - `transformers` libreria di HuggingFace per la gestione dei modelli
 - `datasets` e `evaluate` per la gestione dei dati e delle metriche.
 - `scikit-learn` per il calcolo delle matrici di confusione.
 - `peft` libreria di HuggingFace per l'implementazione di LoRA.

- `numpy` e `matplotlib` per l'analisi dei dati.
- `google` già presente su Colab per la connessione a Google Drive.

Per poter riprodurre i risultati è necessario rieseguire tutte le celle del notebook. A causa dei limiti di utilizzo della T4 GPU di Google Colab, l'esecuzione potrebbe essere interrotta prima del completamento. Per questo motivo è stato collegato Google Drive per il salvataggio dei modelli.

3.4.3 Results

Dopo aver testato differenti tecniche, l'esito migliore è stato raggiunto con la strategia di **Full Fine-Tuning** che ha ottenuto i seguenti risultati sul Validation Set:

- **Accuracy:** 94.15%
- **Macro F1-Score:** 91.86%
- **Training Time:** ~10:30 (5/10 epoche - Early stopping dopo 2 epoche).

La configurazione utilizza un Learning Rate di $2e^{-5}$, un Batch Size di 16, un Weight Decay di 0.01 e un indice di Warmup Steps di 0.1.

3.4.4 Ablation Study: Analysis of Configurations

Lo studio di Ablation è stato svolto in quattro fasi dell'analisi con lo scopo di determinare la configurazione di iperparametri di maggior efficienza:

- **A. Transfer Learning:** i pesi del modello base vengono "congelati", impedendo qualsiasi aggiornamento durante il training. Solo la classification head viene allenata, adattando il modello RoBERTa alle classi del task.
- **B. Full Fine-Tuning:** tutti i pesi del modello base vengono aggiornati durante il training, consentendo al modello di specializzarsi completamente sul task. Questo approccio però richiede risorse computazionali significative.
- **C. LoRA (Low Rank Adaptation):** anziché aggiornare direttamente i pesi del modello, si introducono due matrici di rango ridotto. Solo queste matrici vengono poi addestrate, riducendo drasticamente il numero di parametri.
- **D. In-Context Learning (ICL):** il modello viene utilizzato senza alcun aggiornamento dei pesi. Nel caso few-shot, alcuni esempi etichettati vengono forniti direttamente nel prompt guidando la predizione.

A. Transfer Learning (Allineamento solo della Classification Head)

I parametri del corpo del modello base sono stati "congelati" ad eccezione della testa di classificazione (595, 206 parametri allenabili) ed è stata eseguita una ricerca completa comparando sei configurazioni differenti tra tre valori di Learning Rate $\in \{1e^{-3}, 1e^{-4}, 1e^{-5}\}$ e due dimensioni di Batch Size $\in \{16, 32\}$. Analizzando i risultati della ricerca risulta che, con l'utilizzo di un valore di learning rate maggiore ($1e^{-3}$) si ottiene un addestramento della testa di classificazione più efficiente rispetto a valori inferiori:

- $1e^{-3}$ produce le performance migliori in entrambe le configurazioni di batch size, mentre valori inferiori degradano progressivamente.
- In particolare, $1e^{-5}$ produce risultati identici indipendentemente dalla batch size (34.75% accuracy, 8.60% F1), suggerendo che un learning rate eccessivamente basso impedisce qualsiasi aggiornamento significativo dei pesi della testa di classificazione.

I risultati migliori raggiunti risultano avere un valore di accuracy del 57.95% e un valore di F1-Score del 32.50%, non abbastanza efficienti per il task di text emotion classification rispetto ad altre strategie analizzate in seguito.

Learning Rate	Batch Size	Accuracy	F1-Score
$1e^{-3}$	16	57.95%	32.50%
$1e^{-3}$	32	56.80%	30.66%
$1e^{-4}$	16	51.45%	20.68%
$1e^{-4}$	32	49.65%	19.88%
$1e^{-5}$	16	34.75%	08.60%
$1e^{-5}$	32	34.75%	08.60%

Tabella 9: Risultati delle configurazioni per l’allineamento della testa di classificazione

B. Full Fine-Tuning

La seconda tecnica applicata è stata il Full Fine-Tuning. In questo caso è stato allenato il 100% dei parametri (124, 650, 246) ed è stata eseguita una ricerca completa comparando sei configurazioni differenti tra tre valori di Learning Rate $\in \{1e^{-5}, 2e^{-5}, 3e^{-5}\}$ e due dimensioni di Batch Size $\in \{16, 32\}$ come citato nel paper di riferimento presentato da Yinhan Liu et al. (2019) **yinhan-liu-et-al-2019-roberta**. La configurazione che ha ottenuto i risultati migliori utilizza un valore di Learning Rate di $2e^{-5}$ e una dimensione di Batch Size di 16, raggiungendo un valore di accuracy del 93.10% e un valore di F1 macro del 89.26%. In generale, l’utilizzo di un Learning Rate nell’intervallo $\{1e^{-5}, 2e^{-5}, 3e^{-5}\}$ produce risultati stabili e consistenti, con un valore di accuracy sempre superiore al 92% e un valore di F1-macro maggiore del 88%, confermando la robustezza del Full Fine-Tuning rispetto alla scelta degli iperparametri in questo intervallo.

Learning Rate	Batch Size	Accuracy	F1-Score
$2e^{-5}$	16	93.10%	89.26%
$1e^{-5}$	32	92.60%	88.80%
$2e^{-5}$	32	92.75%	88.40%
$3e^{-5}$	32	92.90%	88.28%
$3e^{-5}$	16	92.30%	88.16%
$1e^{-5}$	16	92.50%	88.03%

Tabella 10: Risultati delle configurazioni di Full Fine-Tuning

C. LoRA

La caratteristica fondamentale di LoRA è il numero di parametri addestrabili. Grazie a questa strategia anziché allenare il 100% dei parametri, ne sono stati allenati solamente 1,479,942 per le configurazioni con un rango di 16 e 2,364,678 per le configurazioni con

rango pari a 32, un utilizzo pari al 1.8% dei parametri totali del modello. Per individuare la configurazione LoRA ottimale, è stata condotta una ricerca su 12 combinazioni di iperparametri tra Rango (16, 32), Learning Rate ($5e^{-5}$, $1e^{-4}$, $5e^{-4}$) e Batch Size (8, 16). La configurazione migliore ($r = 32$, LR $5e^{-4}$, BS 16) raggiunge un’accuracy del 92.65% e un F1-score macro dell’88.46%. In generale, $r = 32$ ottiene risultati leggermente superiori a $r = 16$ a parità di altri iperparametri. Il learning rate risulta essere l’iperparametro di maggior impatto. Valori bassi come $5e^{-5}$ producono performance inferiori, mentre valori di LR di $1e^{-4}$ e $5e^{-4}$ garantiscono risultati migliori. Si può osservare, però, un caso di *training collapse* nella configurazione ($r = 32$, LR $5e^{-4}$, BS 8), che crolla ad un accuracy del 34.75% e un F1 dell’8.60%. Osservando la configurazione identica con BS 16 è possibile notare che non mostra questo problema, suggerendo che batch size ridotte possano amplificare l’instabilità del gradiente se combinati a valori di learning rate elevati e ranghi alti.

Rank	LR	BS	Accuracy	F1-Score
32	$1e^{-4}$	8	92.45%	88.61%
32	$5e^{-4}$	16	92.65%	88.46%
16	$5e^{-4}$	16	92.45%	88.46%
16	$5e^{-4}$	8	92.45%	88.36%
32	$1e^{-4}$	16	92.50%	88.24%
16	$1e^{-4}$	8	92.20%	88.08%
16	$1e^{-4}$	16	91.25%	86.87%
32	$5e^{-5}$	8	91.00%	86.60%
16	$5e^{-5}$	8	90.60%	86.42%
32	$5e^{-5}$	16	90.55%	86.41%
16	$5e^{-5}$	16	88.85%	84.15%
32	$5e^{-4}$	8	34.75%	08.60%

Tabella 11: Risultati delle configurazioni LoRA

D. In-Context Learning (Zero-Shot e Few-Shots)

È opportuno sottolineare che RoBERTa è un modello encoder-only addestrato con un obiettivo di Masked Language Modeling (MLM); a differenza dei modelli decoder generativi, non è stato progettato per l’ICL, e di conseguenza i risultati vanno interpretati tenendo conto di questa considerazione. Con la configurazione zero-shot, il modello raggiunge la performance migliore, con un’accuracy del 51.15% e un F1-score macro del 26.73%. Il divario tra accuracy e F1-score macro suggerisce un comportamento sbilanciato del modello, che tende a predire correttamente le classi più frequenti ignorando quelle minoritarie. Con l’aggiunta di esempi nel contesto le performance peggiorano: la configurazione 1-shot registra il calo più marcato (accuracy 43.53%, F1 21.22%), mentre 3-shot e 5-shot mostrano un lieve recupero, pur rimanendo al di sotto della configurazione zero-shot. Tale andamento è prevedibile data l’architettura di RoBERTa-base. Il modello non è stato ottimizzato per il condizionamento del contesto come fa l’ICL e, di conseguenza, gli esempi aggiuntivi possono interferire con il pattern del prompt atteso dal MLM. È opportuno precisare che, in tutte le configurazioni, non sono stati registrati troncamenti del token <mask>, escludendo quindi risultati legati alla lunghezza del contesto.

Shots	Accuracy	F1-Score
0	51.15%	26.73%
1	43.53%	21.22%
3	45.77%	22.76%
5	46.35%	23.70%

Tabella 12: Risultati dell’In-Context Learning Zero-Shot e Few-Shots

3.5 Analisi degli errori

L’analisi degli errori, eseguita con il modello sul quale è stata applicata la strategia di Full Fine-Tuning con risultati migliori, mostra che il maggior numero di errori di classificazione si concentra su determinate classi di emozioni:

1. **Love e joy:** La classe *joy* risulta tra le più riconosciute dal modello. La classe *love*, invece, registra la performance peggiore con il 19% degli esempi classificati erroneamente come *joy*. Questo errore è semanticamente motivato: testi che esprimono amore condividono spesso un tono positivo e lessico affine a quelli di gioia, rendendo il confine tra le due classi ambiguo. Di seguito vengono mostrati alcuni esempi di errore:

- **Testo:** *‘i feel the need to work on caring’* → **True:** love, **Predicted:** joy
- **Testo:** *‘i was not feeling up to it yet i blamed my fiances deployment for bringing me down’* → **True:** love, **Predicted:** joy
- **Testo:** *‘i feel unimportant and small here lately’* → **True:** love, **Predicted:** joy

Su un totale di 2000 esempi, la classe *love* viene confusa con *joy* 30 volte.

2. **Surprise e Fear:** Anche in questo caso, la classe *surprise* risulta tra le più riconosciute, mentre *fear* è la classe che presenta il maggior numero di errori. Il 12% degli esempi di *fear* viene classificato come *surprise*, e il restante 8% si distribuisce tra *sadness* (4%) e *anger* (4%). La confusione con *surprise* è spiegabile dalla sovrapposizione contestuale delle due emozioni: entrambe implicano una risposta a eventi inattesi, e la distinzione tra timore e stupore può dipendere da sfumature lessicali sottili che il modello fatica a catturare. Di seguito vengono mostrati alcuni esempi di errore:

- **Testo:** *‘i feel tortured’* → **True:** fear, **Predicted:** surprise
- **Testo:** *‘i know that feeling awkward and not having friends in a space contributes to this’* → **True:** fear, **Predicted:** sadness
- **Testo:** *‘im feeling less grumpy after that’* → **True:** fear, **Predicted:** sadness

Su un totale di 2000 esempi, la classe *fear* viene confusa con *surprise* 26 volte.

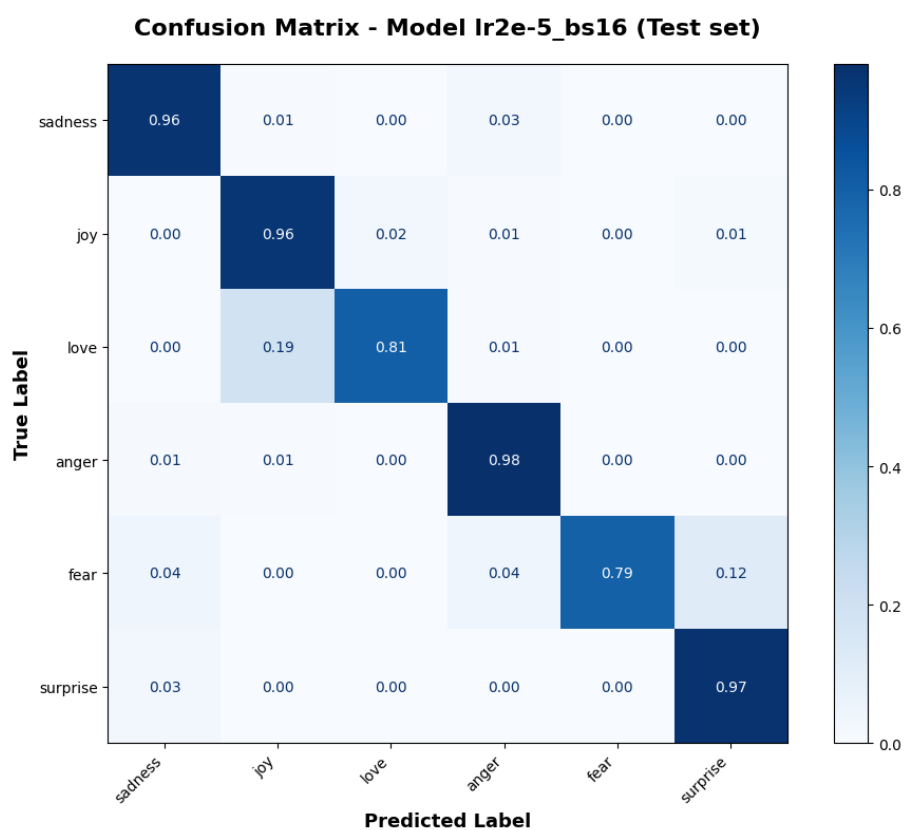


Figura 17: Matrice di Confusione del modello Full Fine-Tuned.

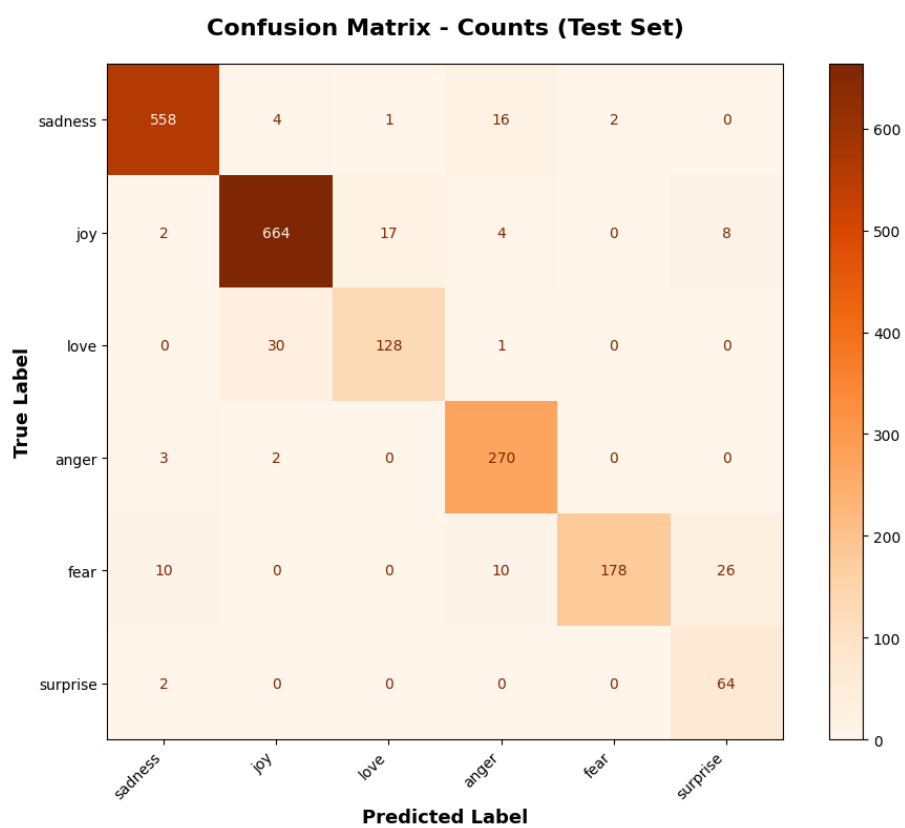


Figura 18: Matrice di Confusione Quantitativa del modello Full Fine-Tuned.

Di seguito vengono riportati gli errori più comuni con la relativa frequenza registrata:

Vera emozione	Predizione	Frequenza
love	joy	30 volte
fear	surprise	26 volte
joy	love	17 volte
sadness	anger	16 volte
fear	anger	10 volte
fear	sadness	10 volte
joy	surprise	8 volte
sadness	joy	4 volte
joy	anger	4 volte
anger	sadness	3 volte

Tabella 13: Frequenza degli errori più comuni registrati per RoBERTa-base

3.5.1 Risultati Finali e Confronto

Il seguente grafico riassume i risultati ottenuti dall'applicazione delle diverse strategie di training del modello RoBERTa-base. Si può subito notare come le strategie di Full Fine-Tuning e LoRA abbiano raggiunto risultati nettamente superiori rispetto alle altre strategie. Nel grafico è stata inserita anche la valutazione di RoBERTa-base con la testa di classificazione casuale che a causa dello sbilanciamento delle classi raggiunge un accuracy del 29.05% e un F1 macro del 7.50%, rispetto a un accuracy del 16% che si otterrebbe con un dataset bilanciato.

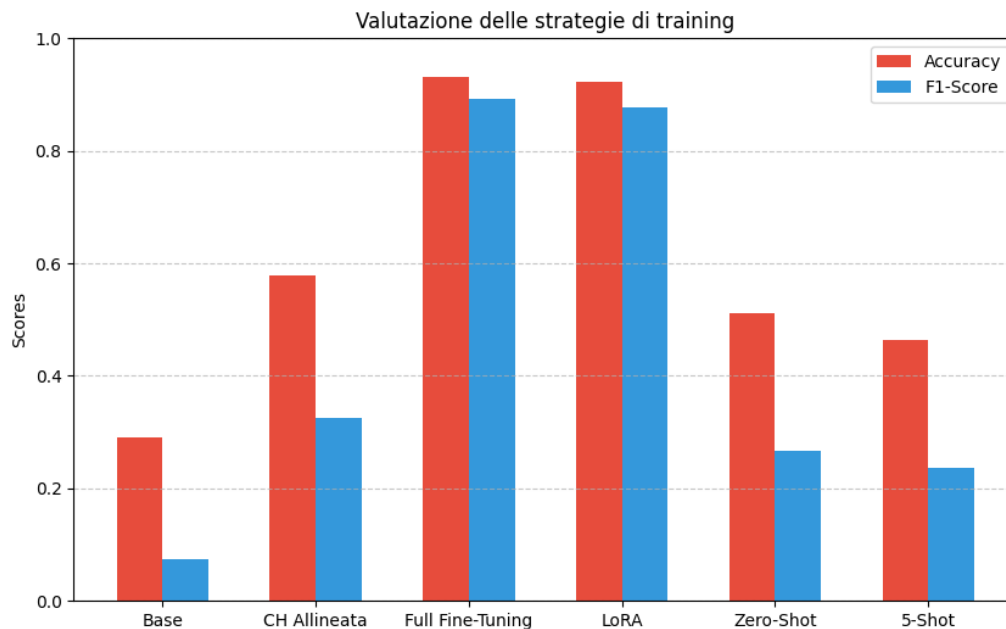


Figura 19: Confronto di Accuracy e F1 macro ottenuti tra le diverse strategie testate.

Per concludere, di seguito si può trovare una tabella che riassume i migliori risultati per ciascuna strategia utilizzata:

Strategia	Accuracy	F1	Conclusioni
Transfer Learning	57.95%	32.50%	Performance limitate. Le predizioni si concentrano sulle classi più frequenti.
Full Fine-Tuning	93.10%	89.26%	Miglior strategia. L'aggiornamento di tutti i pesi consente al modello di specializzarsi sul task.
LoRA	92.35%	87.77%	Ottimo compromesso tra efficienza e performance. Ottimo per risorse limitate
ICL (0-Shot)	51.15%	26.73%	Strategia non adatta a modelli encoder-only.

Tabella 14: Riepilogo finale Ablation Study

3.5.2 Results Discussion

Dai risultati ottenuti è facile determinare che, per il task di text emotion classification utilizzando un modello encoder-only come RoBERTa-base, le due strategie più efficienti sono il **Full Fine-Tuning** e **LoRA**. Il Full Fine-Tuning risulta la strategia che ha ottenuto risultati migliori, ma con un costo computazionale maggiore dovuto alla richiesta di un elevato numero di risorse per l'aggiornamento del 100% dei parametri. In termini di risorse disponibili, LoRA risulta altamente più efficiente rispetto al Full Fine-Tuning, con una perdita del 0.75% sul valore di accuracy e del 1.49% sul valore di F1 macro. I risultati ottenuti sono più che soddisfacenti, in particolare visto lo sbilanciamento delle classi del dataset; con un valore di F1 macro del 89.26% si è dimostrato che il modello risulta in grado di differenziare efficacemente anche le classi meno frequenti come **surprise** e **love**.

3.5.3 Method Validity

I risultati ottenuti con la strategia di Full Fine-Tuning di RoBERTa-base confermano la validità dell'approccio adottato per il task di *text emotion classification*. Al tempo stesso una strategia come l'ICL mostra come non sia adatta a un modello encoder-only per questo tipo di task.

3.5.4 Limitations and Maturity

Il lavoro presenta alcune limitazioni che è opportuno considerare nell'interpretazione dei risultati. Il dataset utilizzato è sbilanciato, di conseguenza, classi come *love* e *fear* sono molto frequenti rispetto ad altre, il che si riflette nel divario tra accuracy e F1-score macro osservato nelle diverse strategie testate. Inoltre, l'utilizzo della strategia di ICL, su un modello non adatto, ha fornito risultati che non sono comparabili per un discorso di efficienza rispetto a modelli decoder generativi come GPT, per i quali l'ICL è stato progettato.

3.5.5 Future Works

Il seguente lavoro potrebbe essere esteso con diversi sviluppi futuri. Un primo contributo significativo sarebbe l'estensione dell'analisi a modelli decoder generativi, come LLaMA o GPT, per una valutazione comparativa dell'efficienza dell'ICL in un contesto architetturale più adeguato. Questo consentirebbe di valutare concretamente il divario di

performance rispetto all'approccio encoder-only utilizzato. Sarebbe, inoltre, interessante estendere l'analisi a modelli multilingua, come XLM-RoBERTa, per valutare lo stesso approccio su testi in lingue diverse dall'inglese e ampliare il campo di applicazione del sistema sviluppato.

3.6 Limitations and Maturity

- **Limiti:** La *Context Window* limitata impedisce l'uso di prompt complessi o catene di ragionamento. Il dataset presenta un sbilanciamento naturale che penalizza leggermente le classi minoritarie (*Surprise*).
- **Maturità:** La soluzione sviluppata, basata su librerie standard industriali (**transformers**, **peft**), ha raggiunto un buon livello di maturità tecnologica, pronta per essere integrata in pipeline di produzione con costi di inferenza contenuti.

3.7 Future Works

Per avanzare ulteriormente nel progetto e superare i limiti identificati, proponiamo le seguenti direzioni di ricerca:

1. **Data Augmentation via Back-Translation:** Per mitigare lo sbilanciamento delle classi (in particolare *Surprise*, che ha pochi campioni), proponiamo di generare dati sintetici utilizzando una pipeline di traduzione andata-ritorno (es. Inglese → Francese → Inglese). Questa tecnica introduce variazioni lessicali e sintattiche naturali mantenendo inalterata la semantica originale, arricchendo il training set con esempi diversificati che rendono il modello più robusto sulle classi minoritarie.
2. **Supervised Contrastive Learning:** Per risolvere l'ambiguità tra classi semanticamente sovrapposte come *Love* e *Joy*, suggeriamo di introdurre una fase preliminare di addestramento con *SupCon Loss*. A differenza della classica Cross-Entropy (che valuta i campioni singolarmente), la loss contrastiva lavora su coppie: forza l'encoder ad avvicinare drasticamente le rappresentazioni vettoriali dei tweet della stessa classe e ad allontanare quelle di classi diverse nello spazio latente. Questo creerebbe confini decisionali più netti prima ancora di addestrare il classificatore finale.
3. **Task-Adaptive Pre-Training:** Invece di passare direttamente dal modello generico (BERTweet) al fine-tuning, proponiamo una fase intermedia di "riscaldamento". Questa consiste nel continuare il pre-training con obiettivo *Masked Language Modeling* (MLM) utilizzando i soli testi del dataset **emotion** (senza etichette). Questo adatta i pesi dell'encoder allo specifico vocabolario, slang e distribuzione linguistica del nostro dataset, ottimizzando la capacità del modello di estrarre feature rilevanti da contesti emotivi ambigui prima di applicare LoRA.